

When Protocol States Mislead: Evidence-Gated Admission and Online Feedback Calibration for LLM-Assisted Stateful Fuzzing

Ketang Chen^a, Xiaolei Ren^{a,*}

^aMacau University of Science and Technology, Avenida Wai Long, Taipa, Macau, China

Abstract

Response-derived states provide inexpensive scheduling feedback for network protocol fuzzing, but state-space expansion need not indicate progress through program code. Large language models (LLMs) introduce another source of uncertain proposals: a plausible request can consume long-term mutation resources without yielding useful executions. We present LoopFuzz as an evidence-calibration controller with two complementary mechanisms. Execute-before-promote admission separates structural validity, response acceptability, and state reachability from independent code and state novelty. Candidates with only state novelty receive bounded provisional validation; durable promotion requires code evidence. Online state-productivity calibration uses discounted Beta–Bernoulli updates at state-selection episodes and combines Thompson samples with frontier exploration. It estimates the usefulness of an observed state proxy, without attempting to recover semantic protocol states. We specify five experimental arms that separate admission from scheduling calibration under matched model policies and resource ceilings, together with information-controlled historical vulnerability rediscovery. The nine-target archive contains AFLNet, ChatAFL, benchmark LoopFuzz, direct-admission LoopFuzz, and calibrated LoopFuzz runs, together with two small gamma sensitivity groups. Run counts and horizons are heterogeneous: some target-arm groups have fewer than ten runs or missing archives, while others contain additional attempts. We report recoverable endpoints and AUCs with observed n , leave unavailable values blank, and do not convert these descriptive results into causal admission or calibration claims. Vulnerability-rediscovery benefits remain unmeasured.

Keywords: Stateful protocol fuzzing, Response-derived state feedback, Evidence-gated admission, Provisional queue, Online productivity calibration

1. Introduction

Stateful protocol fuzzing must discover useful interaction sequences as well as useful message contents. A server can accept a request only after earlier exchanges establish a session, authenticate a client, or create a resource. AFLNet uses responses to construct an inferred protocol state machine (IPSM), making protocol-visible feedback available without a target-specific internal-state extractor [1]. This deployment advantage comes with an abstraction boundary: a response code or sequence is evidence about observable behavior, rather than an identifier of the server’s complete semantic state.

StateAFL already identifies limitations of response-derived state abstractions and uses memory observations to infer server states [2]. The fact that responses can misrepresent internal progress is therefore established background. Our question is how a fuzzer retaining a response-visible abstraction should decide, during a campaign, whether a particular state signal is useful enough to justify further

exploration. A new IPSM state edge can accompany a previously unseen code branch, reproduce an already explored rejection path, or distinguish responses after the measured code surface has saturated. These outcomes should not receive identical evidential weight.

LLM-assisted protocol fuzzing makes this distinction consequential for queue management. ChatAFL demonstrates that language models can derive protocol structures and propose interaction sequences [3]. A generated request is nevertheless a hypothesis about both message structure and live session conditions. If a proposal is retained indefinitely because it produces a new response-derived state, the fuzzer may repeatedly mutate an input whose apparent novelty has little downstream code value. Durable queue membership represents a continuing allocation of execution resources.

Three target families motivate the investigation. Forked-daapd exposes DAAP/DACP behavior through HTTP status responses, so response distinctions may overstate useful code progress. Lighttpd1 can execute different code paths under the same HTTP status, so response novelty may understate code progress. LightFTP motivates examining continuing IPSM growth after code-coverage saturation. These are hypotheses inherited from earlier development ob-

*Corresponding author: Xiaolei Ren.
Email addresses: 3250006913@student.must.edu.mo (Ketang Chen), xiren@must.edu.mo (Xiaolei Ren)

servations, not conclusions imposed on the present archive. Section 2 separates them from the available measurements.

LoopFuzz addresses this problem through evidence-gated admission and online state-productivity calibration. It first checks a proposal’s structure, then executes a bounded trial and separates response acceptability, state reachability, code novelty, and IPSM novelty. Code-supported candidates can enter the durable queue. State-only candidates receive a provisional validation budget and must earn durable promotion through subsequent code evidence. At the scheduling layer, a discounted productivity estimate conditions investment on the observed relationship between selected response states and later code progress.

This work makes three contributions, with their current evidence status stated explicitly.

1. **Evidence-gated queue admission.** We specify an execute-before-promote controller that distinguishes reject, provisional, and durable outcomes. It separates state novelty from code evidence and makes the lifetime and descendant cost of provisional entries auditable.
2. **Online calibration of state productivity.** We formulate response-state utility as the probability of code progress in a bounded state-selection episode. Discounted Beta–Bernoulli updates and Thompson sampling instantiate the controller; these established estimation and selection methods are not claimed as new algorithms.
3. **A controlled empirical framework with partial evidence.** We define candidate-level and episode-level measurements, five arms, launch accounting, and information-controlled rediscovery on vulnerable/patch-paired targets. We audit the supplied archive and identify the missing evidence for causal admission and calibration claims.

The available evidence does not justify describing the third contribution as a completed five-arm evaluation. Preserving this distinction allows the design, implementation, and later experiments to be evaluated against the same explicit contract.

2. Background and Motivation

2.1. Response-visible state and independent code feedback

Let $\mathcal{M}_t = (Q_t, \hat{\Delta}_t, q_0)$ be the IPSM observed by time t , where Q_t contains response-derived state identifiers, $\hat{\Delta}_t \subseteq Q_t \times Q_t$ contains observed IPSM state edges, and q_0 denotes the initial connection state. The abstraction can alias different internal states and separate executions that reach the same code. It can also depend on history, asynchronous replies, and reset behavior. It is not assumed deterministic or semantically complete.

Let $\mathcal{B}_t$ be the covered code-branch set under a fixed instrumentation and replay pipeline. An IPSM state-edge

increment changes $\hat{\Delta}_t$; a code-branch increment changes $\mathcal{B}_t$. These objects have distinct units. An AFL bitmap observation must also distinguish a newly encountered code edge from a hit-count bucket change. Neither bitmap slots nor response-state identifiers are interchangeable with source-level code branches.

2.2. Three falsifiable mismatch cases

Table 1 defines the three motivating cases; Fig. 1 shows the corresponding observations available from the supplied archive. Establishing a mismatch mechanism requires comparable builds, a shared observation horizon, and execution-level or time-resolved evidence. Endpoint differences alone do not establish which state caused a difference. The partial archive must be allowed to contradict the earlier negative-target narrative.

The desired controller must accommodate both overestimation and underestimation. Rejecting every state-only candidate could lose useful bridges; promoting every such candidate would treat an imperfect proxy as sufficient evidence for permanent resource allocation. A provisional queue expresses the intermediate conclusion that an input deserves limited descendant validation while its longer-term value remains unproven.

The scope is textual or semi-textual request-response protocols with usable parsers, response extractors, and code feedback. Response-visible state extraction avoids a target-specific internal-state adapter, but the complete controller remains greybox because productivity requires code feedback. It is not a pure black-box system. Encrypted or opaque binary protocols require suitable construction, response, and code-observation adapters before evaluation.

3. Related Work and Novelty Boundary

Stateful greybox fuzzing studies how to expose, infer, and schedule protocol state [1, 2, 4, 5]. AFLNet supplies a response-derived IPSM, whereas StateAFL and NSFuzz use different internal observations. LoopFuzz retains the response-visible interface and studies its instrumental reliability under an independent code-progress criterion. It does not claim to be the first system to identify state aliasing or misleading responses.

LLM-assisted generation addresses structured inputs and interaction histories [3, 6, 7]. Grammar-aware and documentation-guided methods likewise improve input structure [8, 9]. Better proposals still need execution evidence before acquiring long-term queue resources. Our controlled ChatAFL comparison preserves its scheduling lineage rather than replacing its scheduler.

State and seed scheduling already use adaptive resource allocation. The Bandit’s States studies bandit-based protocol-state selection; T-Scheduler applies Thompson-sampling ideas to fuzzing scheduling; SSGFuzz studies state

Table 1: Motivating mismatch hypotheses and their present evidential boundary.

Target	Hypothesis	Required observation	Current interpretation
Forked-daapd	Overestimation: IPSM state-edge growth without downstream code gain.	Selected-state episodes with growing IPSM structure and low code reward.	The new archive does not reproduce the older endpoint code regression.
Lighttpd1	Underestimation: code exploration within a shallow response abstraction.	Code-branch gain with little IPSM state-edge gain, joined to executions.	Both native endpoint metrics increase; their aggregate changes do not establish hidden productive states.
LightFTP	Saturation mismatch: response novelty persists after code progress plateaus.	Aligned code-branch and IPSM state-edge trajectories before and after saturation.	Near-equal native code endpoints motivate the test; current-build trajectories are still required.

significance in protocol fuzzing.¹ These systems delimit the novelty claim: a bandit or productivity score is not itself a contribution. The distinction here is the combination of a response-derived proxy, an independent code-progress target, two-tier LLM-candidate admission, and auditable episode evidence.

Benchmark methodology requires repeated trials, comparable instrumentation, explicit budgets, and careful failure interpretation [10–12]. Historical vulnerability benchmarks such as Magma motivate distinguishing execution reaching bug-related code from execution satisfying a trigger condition. We adopt that distinction for network-reachable vulnerable/patch pairs; crash-file counts are not the unit of security success.

4. Evidence-Calibrated Controller

This section specifies the controller contract. Section 5 identifies where the inspected implementation and archive do not yet satisfy it. A mathematical design rule is not evidence that a completed campaign obeyed that rule.

4.1. LLM proposal interface

The model receives permitted RFC fragments, seed traces, and observed interaction summaries and returns candidate requests or bounded scheduling hints. Each candidate has a persistent identifier, source state, intended target/frontier, and links to its prompt and raw reply. Message-field hypotheses remain proposals rather than verified protocol grammars.

The information policy excludes CVE identifiers, security advisories, exploit inputs, and patch contents. It fixes which documentation and live responses may be supplied. Identical information policy does not imply identical prompt bytes after adaptive campaigns diverge; prompt and reply hashes expose that distinction.

¹Reference completion required before submission: full bibliographic records for The Bandit’s States, T-Scheduler, SSGFuzz, and Magma remain unverified. Their names and comparison roles are supplied by the revision brief; no author, year, venue, or DOI has been inferred.

Request-producing outputs pass through admission. Scheduling-only outputs require schema, state-ID, and seed-ID checks and cannot insert durable inputs. Parsing or transport failure does not create a positive admission decision. All attempts, retries, failures, and successful replies enter the declared call/token accounting.

4.2. Admission predicates and evidence separation

For candidate x , $P(x)$ checks schema, message structure, required fields, lengths, and terminators before execution. A bounded trial then evaluates response acceptability $U(x)$ and reachability $R(x)$: arrival at a declared target/frontier or escape from a recorded rejection sink. Table 3 gives operational evidence and required reason codes.

HTTP/FTP 4xx/5xx classes are rejection priors, not a universal semantic definition. An adapter must specify exceptions before a run and retain the response and reason code. Timeout, reset, disconnect, or an unparseable response gives no positive U evidence. A valid response alone is insufficient for R . Security-oracle artifacts are retained independently even when normal request admission fails.

Gain is split into

$$G_{\text{code}}(x) = \mathbf{1}\{\text{new code coverage or a favored code-coverage entry}\}, \quad (1)$$

$$G_{\text{state}}(x) = \mathbf{1}\{\text{new IPSM node, state edge, or path}\}. \quad (2)$$

Favored status must be grounded in code coverage; merely being queued or state-novel does not qualify. Source-level code branches are the main evaluation surface. If runtime instrumentation observes code edges instead, that reward definition must be frozen and reported separately. Hit-count novelty is a separate sensitivity condition. Pre-trial coverage and IPSM snapshots establish which gains are new.

4.3. Provisional and durable queues

Let Q_{prov} contain temporary candidates and Q_{dur} contain long-term mutation seeds. Admission has three out-

Descriptive native endpoints

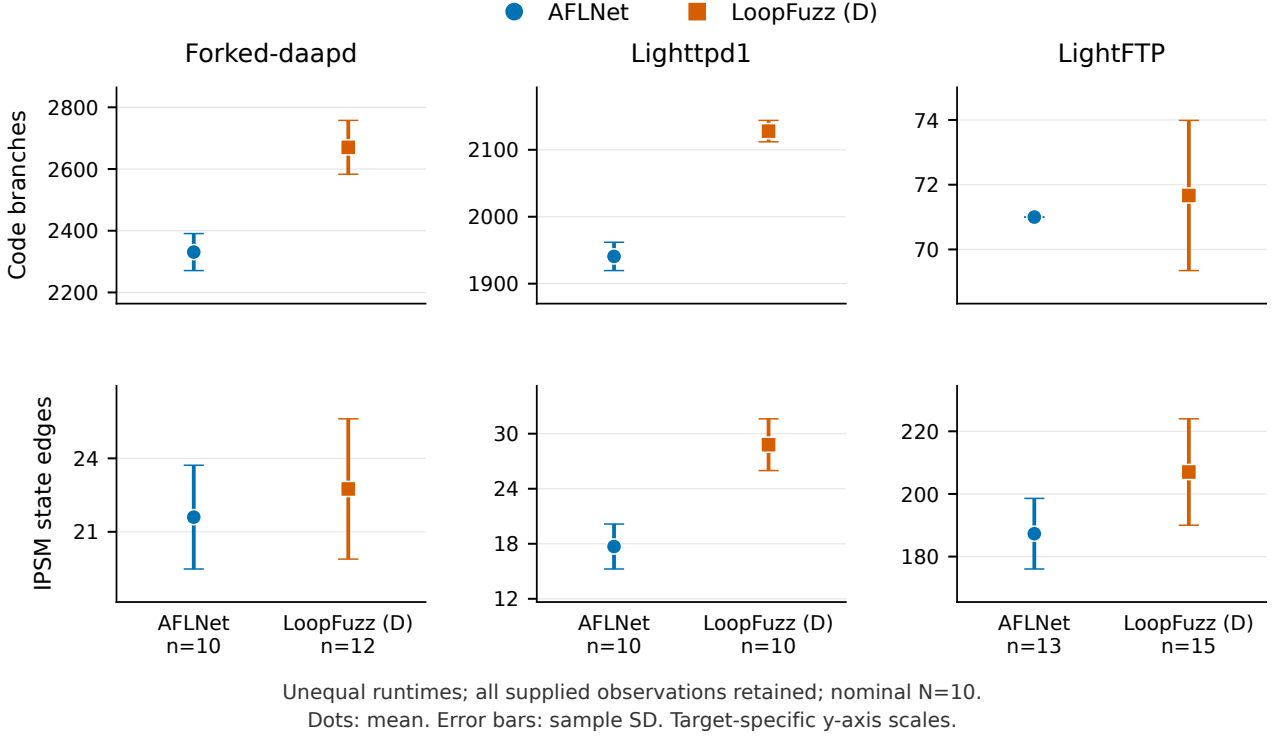


Figure 1: Observed native terminal code-branch and IPSM state-edge counts for the three motivating targets (mean and sample standard deviation; observed run counts are labeled; nominal target $N = 10$). All runs, including interruptions, are retained. Runtime and provenance differences prevent a matched-horizon causal interpretation. This figure tests the earlier narrative against current observations: it does not assert that all three hypothesized mismatch types were reproduced.

comes:

$$D(x) = \begin{cases} \text{durable,} & P \wedge U \wedge R \wedge G_{\text{code}}, \\ \text{provisional,} & P \wedge U \wedge R \wedge \neg G_{\text{code}} \wedge G_{\text{state}}, \\ \text{reject,} & \text{otherwise.} \end{cases} \quad (3)$$

IPSM novelty alone is never sufficient for durable promotion. These logical conditions do not assume statistical independence among predicates.

A provisional entry receives at most $H = 64$ descendant executions and a local validation deadline of 30 seconds, whichever is exhausted first; the proposed live provisional capacity is 64. These initial settings are checked in the pilot and frozen before formal runs. Budget checks occur before each descendant execution. Exhaustion expires the entry from active scheduling while preserving its audit record. Campaign termination before the validation horizon is censoring, not unproductive expiration.

A descendant supplying qualifying code evidence permits promotion of the replayable parent sequence and productive descendant, subject to the same structural, response, and reachability checks. The record identifies the first productive descendant and the candidate whose status changes. A global code-discovery event is credited once through execution/lineage IDs, not repeatedly to multiple

ancestors.

Productivity estimates select where to propose and continue exploration; they do not independently relax Equation 3. Fixed and calibrated arms use identical admission rules and provisional budgets. Calibration thus affects admission outcomes indirectly through exploration opportunities, preserving the causal scheduling comparison.

4.4. State-selection episodes and independent reward

An episode begins when the scheduler selects response-derived state s and a seed capable of reaching it. It receives one fixed execution-energy budget. Individual executions within the episode do not generate repeated posterior updates. Mutation counts, reset failures, elapsed time, and budget exhaustion are logged. Infrastructure-interrupted episodes are incomplete, rather than silently assigned reward zero.

For completed episode t targeting s_t , let ΔB_t count previously unseen code branches discovered within that episode. The main reward is

$$r_t = \mathbf{1}\{\Delta B_t > 0\}. \quad (4)$$

A declared code-edge instrumentation variant is admissible, but IPSM novelty is excluded. Favored descendants are a secondary sensitivity reward. Equal energy prevents a

Table 2: Prior work and novelty boundary. This compares mechanisms, not measured performance.

Prior work	Established focus	Present boundary
AFLNet; StateAFL; NSFuzz	Response-derived or internal-state feedback.	Calibrate a retained response proxy without claiming semantic-state recovery.
ChatAFL; LLM synthesis	Protocol-informed message/sequence proposals.	Execute-before-promote and bounded provisional validation.
The Bandit’s States	Bandit-based protocol-state selection.	Bandit formulation is prior art; independent reward and queue evidence are the focus.
T-Scheduler	Probabilistic adaptive fuzzing scheduling.	Beta estimation and Thompson sampling are reused components.
SSGFuzz	State-significance-guided protocol fuzzing.	Distinguish state importance from calibration against code productivity.
Magma; evaluation methodology	Ground-truth bug conditions and repeated evaluation.	Information-controlled network rediscovery with minimal security-patch pairs.

Table 3: Admission predicates relative to the recorded pre-trial snapshot.

Predicate	Positive evidence	Example failure	Required audit fields
P	Schema and local structural checks pass.	Missing field, invalid length or delimiter.	Candidate ID, parser/schema version, reason, hypothesis hash.
U	Adapter-defined acceptable response.	Rejection, timeout, reset, drop, unparseable reply.	Response code/hash, adapter, flags, exception rule.
R	Target/frontier reached or documented sink escape.	Repeats a sink or misses the destination.	Source, intended target, observed state sequence.
G_{code}	New code branch/code edge or code-supported favored entry.	IPSM-only novelty.	Pre/post code observations, novelty class, favored reason.
G_{state}	New IPSM node, state edge, or path.	Known response-state structure.	Pre/post IPSM observations and new transitions.

state appearing more productive merely because it receives more executions. Trial execution, ordinary mutation, and provisional validation use distinct event types to avoid duplicate updates.

θ_s denotes the probability of positive code reward conditional on selecting s under the episode policy and current campaign phase. It is instrumental utility, not the probability that s is a true semantic state. U continues to mean response acceptability; utility is never denoted $U(s)$.

4.5. Discounted productivity estimates

Each state starts with $\alpha_s = \beta_s = 1$ and

$$\theta_s \sim \text{Beta}(\alpha_s, \beta_s), \quad \hat{\theta}_{s,t}^{\text{pre}} = \frac{\alpha_{s,t}}{\alpha_{s,t} + \beta_{s,t}}. \quad (5)$$

For a completed selected-state episode:

$$\alpha_s \leftarrow 1 + \gamma(\alpha_s - 1) + r_t, \quad (6)$$

$$\beta_s \leftarrow 1 + \gamma(\beta_s - 1) + (1 - r_t). \quad (7)$$

The proposed discount is $\gamma = 0.995$, with sensitivity values $\{0.99, 0.995, 1.0\}$. Parameters are frozen after the pilot and before inspecting formal results. At $\gamma = 1$, updates reduce to cumulative Beta–Bernoulli counts. For $\gamma < 1$, excess prior counts are discounted to respond to changing productivity as coverage saturates.

The term productivity posterior denotes this discounted working model. With discounting it is not the exact posterior of a stationary Bernoulli process. Discounting occurs on completed episodes for the selected state; unselected states do not automatically age in wall-clock time. The update clock and cold start are therefore explicit applicability limits.

Every calibration metric uses the pre-update mean recorded before reward. Post-update predictions would leak the outcome into their own evaluation. Thompson samples are recorded separately and are not substituted for predicted Bernoulli probabilities in reliability calculations.

4.6. Calibrated scheduling

The topology term favors underexplored observed states:

$$F(s) = \begin{cases} 4, & s \text{ is newly reachable or } \deg^+(s) \leq 1, \\ 2, & 1 < \deg^+(s) \leq 3, \\ 1, & \deg^+(s) > 3. \end{cases} \quad (8)$$

Only reachable states with eligible seeds participate. $\deg^+(s)$ counts outgoing IPSM state edges, not code branches. Let $B(s)$ denote the inherited base weight shared by C, D, and E. Define

$$\text{Frontier}(s) = B(s)F(s). \quad (9)$$

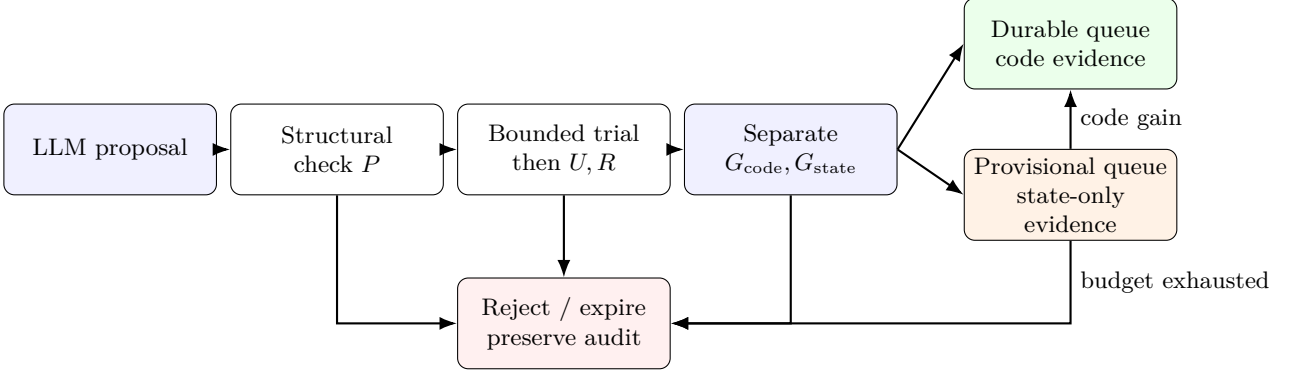


Figure 2: Controller contract: P precedes execution; U/R and distinct code/IPSM evidence precede queue disposition. Native saving must not bypass the boundary. Security-oracle artifacts are preserved separately.

Keeping this factor and any warm-up policy fixed prevents calibration from simultaneously changing the baseline scheduler. Draw

$$\tilde{\theta}_s \sim \text{Beta}(\alpha_s, \beta_s) \quad (10)$$

and calculate

$$S(s) = \text{Frontier}(s)[\epsilon + (1 - \epsilon)\tilde{\theta}_s], \quad \epsilon = 0.1. \quad (11)$$

Select a state with probability proportional to $S(s)$, recording the pseudorandom seed. This is Thompson-sampled weighted selection, not an argmax policy. Under a finite eligible set, the positive floor gives nonzero selection probability, not a fixed number of selections.

The fixed-policy comparator retains the original penalty:

$$S_D(s) = \text{Frontier}(s)\rho_{\text{fixed}}(s). \quad (12)$$

Its cached queue-yield ratio $p_s = D_s/(N_s + 1)$ counts interesting queued paths D_s per state selections N_s . It is not a calibrated Bernoulli probability and can reflect state-only novelty or unequal execution energy. E replaces only ρ_{fixed} with the calibrated factor in Equation 11, preserving the shared base, admission, prompts, triggers, repair settings, and caps. The source implementation’s integer score conversion and initial round-robin cycles require explicit freezing and verification before comparison.

4.7. Intervention policy and optional repair

Plateau assistance uses a frozen definition, cadence, counters, and retry policy. The legacy trigger combines IPSM state-edge growth with an unproductive streak. If retained, its dependence on the proxy remains common to the causal arms and is reported; calibrating scheduling does not automatically calibrate triggering.

Repair is disabled in primary C/D/E comparisons. Counterexample-guided local repair remains an optional implementation feature evaluated diagnostically. E-with-repair versus E-without-repair records failure type, counterexample, parent repair ID, before/after hypothesis hashes, changed fields, validation, post-repair predicates,

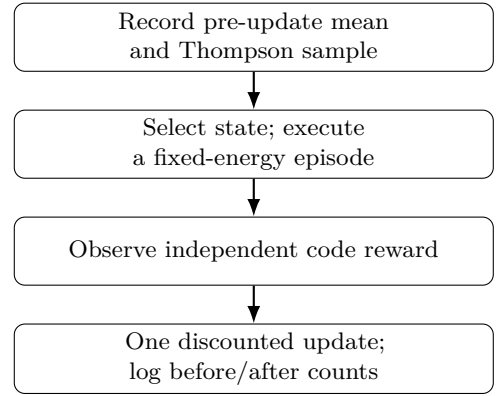


Figure 3: Productivity calibration: prediction precedes reward. IPSM state-edge growth is diagnostic and excluded from the main reward.

descendant gain, and token/latency cost. Sparse events or illustrative examples do not establish an effect. An additional contribution would require sufficient events, reproducible code gains, and independent statistical support across several targets.

5. Implementation and Evidence Status

The source includes split gain fields, provisional-queue machinery, and a posterior-scheduling switch. Their presence does not establish that archived runs exercised or correctly enforced the intended mechanisms. Run configuration, execution ordering, and event semantics must be checked together.

Several differences are material. The candidate path invokes native execution before evaluating the post-trial gate; native execution can already save a code-novel input. A later U/R failure does not undo that insertion, so the inspected implementation does not strictly enforce Equation 3. Native bitmap novelty includes hit-count changes, rather than exclusively new code branches. Provisional accounting checks budgets after a complete fuzzing round and can overshoot the nominal limits.

Algorithm 1: Evidence-gated candidate processing

Input: Candidate x , target, immutable pre-trial snapshot

- 1 Record candidate, prompt, reply, and lineage identifiers
- 2 **if** $\neg P(x)$ **then**
- 3 Record structural rejection; stop processing x
- 4 Execute one bounded trial without durable queue insertion
- 5 Record $U, R, G_{\text{code}}, G_{\text{state}}$ and reason codes
- 6 **if** $P \wedge U \wedge R \wedge G_{\text{code}}$ **then**
- 7 Promote x to Q_{dur}
- 8 **else if** $P \wedge U \wedge R \wedge \neg G_{\text{code}} \wedge G_{\text{state}}$ **then**
- 9 Insert x into Q_{prov}
- 10 **while** *budget, validation deadline, and campaign time remain* **do**
- 11 Check budget before the next descendant execution
- 12 Execute and record one descendant
- 13 **if** *qualifying code gain and validation pass* **then**
- 14 Record first productive descendant; promote; end validation
- 15 Record expiration or campaign censoring if still provisional
- 16 **else**
- 17 Record rejection
- 18 Preserve security-oracle evidence separately

In addition, trial IPSM novelty is computed from graph updates tied to queue saving. In the inspected ordinary gated path, an unsaved candidate does not independently update that graph before the state-only check, whereas a native save already supplies code evidence. This ordering obstructs the intended state-only provisional branch. Zero provisional records therefore cannot be interpreted as successful filtering or proof that provisional validation is unnecessary.

Existing P flags can reflect action-schema acceptance rather than complete wire-format validation. The R implementation also admits generic parsed transitions without requiring the intended target/frontier. These are weaker conditions than the design definitions. Table 4 summarizes the boundaries. This manuscript revision does not change fuzzer code; it separates the proposed contract from the implementation evidence.

5.1. Append-only event contract

Six core streams record run configuration, candidate generation, admission trials, provisional lifecycle, state-selection episodes, and bug-oracle events. Repair adds a seventh optional stream. Table 5 specifies joinable fields. Missing fields are not replaced with zero.

Before formal runs, candidate-event completeness must reach at least 99.9%; no trial may lack a candidate ID, no

posterior update may lack an episode ID, and every promotion must be reconstructable. Completeness denominators include failed generation and parsing paths, not only successful candidates. File presence, join completeness, and agreement between logged disposition and actual queue state are separate checks.

6. Experimental Protocol

This protocol is prospective for unfinished confirmatory experiments. The supplied archive does not establish a dated preregistration, parameter freeze, or completed pilot. We do not retrospectively call it preregistered. A version, timestamp, and checksum must be frozen before the remaining runs, with subsequent deviations recorded. Development and pilot observations remain separate.

6.1. Research questions

1. **RQ1: State/code alignment.** How reliably does response-derived state novelty predict downstream code progress across targets and campaign phases? Measure IPSM state-edge/code-branch alignment, episode productivity, calibration, and boundary cases.
2. **RQ2: Admission.** Does execute-before-promote reduce durable queue pollution and improve downstream productivity under matched proposal and scheduling conditions? Compare D with C.
3. **RQ3: Calibration.** Does online productivity calibration improve scheduling over a fixed response-state heuristic? Compare E with D.
4. **RQ4: Historical vulnerability rediscovery.** Does the controller improve reproducible rediscovery under independent reached/triggered oracles and patch validation?
5. **RQ5: Cost and applicability.** What runtime, throughput, and LLM costs arise, and under which target characteristics does the method fail to help?

The directional hypotheses are improved downstream productivity for D over C and improved code-branch AUC for E over D. Regressions, null results, and uncertainty are reported per target rather than hidden in pooled means or replaced by increased IPSM state-edge counts.

6.2. Five arms and causal comparisons

C/D share model configuration, prompt construction, trigger, repair setting, scheduler, and budget policy; the intended intervention is admission. E/D share admission and differ only in state selection. Controlled ChatAFL retains its scheduling lineage; replacing it with the Loop-Fuzz scheduler would change the baseline. B-A is not an isolated admission comparison.

Proposal control has two scopes. A paired candidate diagnostic evaluates the same recorded output from the same target/campaign snapshot under direct and gated admission with identical descendant budgets. It supports

Table 4: Controller contract versus inspected implementation and available evidence. Source findings describe the current checkout; archive/source identity is not assumed without a commit or binary hash.

Contract	Current boundary	Evaluation consequence
Gate precedes durable promotion	Native saving may precede post-trial U/R checks.	Disposition/reason text does not certify strict queue protection.
Independent state-only trial evidence	IPSM update depends on saving in the inspected path.	Zero provisional events cannot establish a low state-only rate.
Full structural and reachability checks	P can mean action schema; R accepts generic state transitions.	Audit exact parser and target-hit evidence.
Main reward is new code coverage	Bitmap novelty includes hit-count changes; some named branch fields count bitmap slots.	Keep offline branches, bitmap slots, and save events distinct.
Fixed-energy episode	Existing rounds vary mutation energy and may include zero-execution attempts.	Raw episode productivity is not equal-opportunity productivity.
Strict provisional budget	Limits checked after a whole fuzzing round.	Nominal 64/30-second thresholds are not enforced maxima.
Verified run configuration	Start event is written before some environment flags are parsed.	Cross-check launcher, end records, and effective execution path.
Equal total call/token caps	The logged 64 cap concerns plateau use; token-cap logging does not prove enforcement.	No existing total-budget fairness claim.
Repair disabled in C/D/E	Code defaults and activation differ from the prospective configuration.	Legacy labels do not certify the complete D-arm contract.

a candidate-level counterfactual. Independent online campaigns diverge in queue and state history, so matching policies cannot guarantee identical prompt bytes or model outputs throughout. Recorded-action replay supports parser checks, predicate replay, snapshot comparisons, and provider-free artifact inspection; it does not make a replayed campaign equivalent to live adaptation.

6.3. Targets, builds, and resource controls

The primary design contains nine servers: four FTP servers, Exim (SMTP), Live555 (RTSP), Kamailio (SIP), Forked-daapd (DAAP/DACP over HTTP), and Lighttpd1 (HTTP). Exact build metadata unavailable for a verified comparison remains blank in Table 7. Hardware settings and version pins are not carried over from the old manuscript without evidence linking them to the present runs.

Each causal comparison uses the same source revision, flags, instrumentation, seeds, replay procedure, and reset policy. Launch order is randomized within host blocks and balanced across machines and periods. Runs record fuzzer seeds, quotas, measured CPU use, throttling, and execution rates. Container quotas alone do not establish equal CPU time. Startup, model waiting, admission trials, provisional descendants, and cleanup enter the resource ledger.

Each arm initially receives 10 independent 24-hour runs per target; C/D/E receive 10 more, reaching 20 each. For nine targets and one allocated CPU per run, the stages require 10,800 and 6,480 CPU-hours, totaling 17,280 before overhead/failures. These are planned resources, not measured usage. A four-target pilot covers Forked-daapd,

Lighttpd1, LightFTP, and ProFTPD or Pure-FTPd with five arms, three runs, and four to six hours each. Pilot observations are excluded from formal inference.

The intention-to-run ledger includes planned, launched, completed, timeout, target/fuzzer crash, infrastructure failure, extraction failure, excluded, and rerun states, with reasons and linked attempts. Poor coverage is never an exclusion criterion. Only predefined infrastructure failures permit reruns, and original launches remain visible. An interrupted run is not silently extended to 24 hours. It contributes a declared failure outcome or censored observation according to the estimand, with sensitivity analysis; completed-run-only results are not intention-to-run results.

6.4. LLM fairness and configuration

The main design uses equal policy and equal ceilings. All LLM arms must use identical model configurations, intervention policies, and resource ceilings. Actual calls and token usage are reported because adaptive trajectories can trigger different numbers of interventions. Table 8 does not invent unfrozen settings.

An output-token cap per request is not a total campaign ceiling. A zero legacy token-cap field must be interpreted through its actual enforcement, not reported as equal-token control. Dollar costs require auditable usage and applicable prices; unknown charges remain blank.

Secondary comparisons report code-branch gain per 1,000 total tokens, per LLM call, at a common cumulative token budget, and at a common call count. Security costs use CPU-hours and 100,000-token units. Accounting includes unsuccessful generations, retries, validation, and optional repair. A fixed intervention schedule provides a

Table 5: Required logging schema. Each event also includes run ID, schema version, event sequence, and timestamp. Required fields are not claimed to be fully present in the current archive.

Stream	Minimum additional fields
run_config	Arm; target/fuzzer commits; image digest; model/version/endpoint; temperature, top- p , penalties; call/token caps; seed; CPU/memory quota; start/end; termination reason.
candidate_event	Candidate/prompt IDs; prompt/raw-reply hashes; raw candidate or artifact path; action; input/output tokens; source/requested states; generation timestamp; parent candidate.
admission_trial	Candidate/trial IDs; $P/U/R$ and reasons; response code/hash; timeout/reset/drop; state sequence; target/frontier reached; pre/post IPSM nodes/state edges and code coverage; favored reason; latency; disposition.
provisional_event	Stable candidate/provisional IDs; budget; actual descendant executions/code gain; first productive descendant; conversion timestamp; promotion latency; expiration/censoring reason.
state_selection_episode	Episode/state/seed IDs; pre-update $\alpha, \beta, \hat{\theta}$; Thompson sample; frontier/final score; mutations; code-branch/IPSM state-edge gain; reward; post-update α, β ; latency; completion status.
bug_event	Anonymous oracle ID; reached/triggered; first trigger time; reproducing input; vulnerable/patched result; sanitizer evidence; root-cause signature. CVE identity remains isolated metadata.
repair_event (optional)	Repair/parent IDs; failure/counterexample; before/after hypothesis hashes; changed field; validation before/after; post-repair predicates/descendant gain; tokens/latency.

Table 6: Confirmatory arm matrix. Repair is disabled in C/D/E. Names specify required configurations, not inferred identities from archive filenames.

Arm	LLM	Admission	Scheduler	Role
A: AFLNet	No	Native AFLNet	AFLNet	Non-LLM baseline.
B: Controlled ChatAFL	Yes	ChatAFL direct/open-loop	ChatAFL lineage	B-A: total controlled LLM-lineage change.
C: LoopFuzz-direct	Yes	Parseable candidates directly durable	Fixed LoopFuzz	Admission counterfactual.
D: LoopFuzz-gated-fixed	Yes	Evidence gate + provisional queue	Same fixed LoopFuzz	D-C: admission effect.
E: LoopFuzz-gated-calibrated	Yes	Identical to D	Productivity calibration	E-D: scheduling calibration.

cost-control sensitivity, not a replacement for the adaptive main policy. Ratios with zero denominators are undefined.

6.5. Mechanism and calibration metrics

For every target/arm, report candidate generation, $P/U/R$ first-failure distribution, direct durable and provisional rates, conversion, expiration, promotion latency, queue size, trial latency, descendant executions, code gain per candidate/token, episode productivity, and throughput. Denominators and incomplete/censored events accompany rates.

Downstream productivity uses a fixed descendant horizon rather than admission gain. For a declared cohort $\mathcal{C}$ of N candidates enrolled before their outcomes are known, let $Y_i(H) = 1$ if a later descendant produces new code coverage within H executions, and 0 if the full horizon is observed without gain. With complete observation, define

$$\text{Productivity@ } H = \frac{1}{N} \sum_{i \in \mathcal{C}} Y_i(H). \quad (13)$$

Initially $H = 64$. The admission trial is excluded. A positive outcome can be established early; a negative requires the complete horizon. With n_+ known positives and $n_?$ censored unresolved candidates, report the cohort bounds $[n_+/N, (n_+ + n_+)/N]$ rather than deleting incomplete negatives from the denominator. The primary paired diagnostic

reserves the same full validation horizon for both controllers; incomplete online cohorts are reported with these bounds and censoring counts. Durable and provisional cohorts are distinct; post-promotion productivity uses a fresh horizon. Thus neither admission gain nor selective observation mechanically raises measured productivity.

Avoided queue pollution requires C-D evidence. In the paired diagnostic, identify direct-arm durable candidates without downstream gain and compare their retention under gating. D-arm rejections are not automatically pollution. A random, predeclared sample of rejected candidates receives shadow validation to estimate false negatives, with inclusion probabilities, snapshots, lineage, and cost recorded.

For complete episodes, $p_t = \hat{\theta}_{s_t, t}^{\text{pre}}$ predicts later reward r_t . The Brier score is

$$\text{BS} = \frac{1}{n} \sum_{t=1}^n (p_t - r_t)^2. \quad (14)$$

Expected calibration error (ECE) uses ten fixed equal-width probability bins and reports counts, mean predictions, and observed frequencies. Empty bins stay empty. Reliability plots and AUPRC accompany the score; with no positive episodes AUPRC is undefined. Compare Brier score to a prequential base-rate predictor because rare rewards can make uninformative near-zero predictions look accurate.

Table 7: Target/protocol/build matrix. Blank build fields await exact commit, image, coverage, and reset manifests.

Target	Protocol	Commit/version	Image digest	Coverage build / reset hook
LightFTP	FTP			
bftpd	FTP			
ProFTPD	FTP			
Pure-FTPd	FTP			
Exim	SMTP			
Live555	RTSP			
Kamailio	SIP			
Forked-daapd	DAAP/DACP			
Lighttpd1	HTTP			

Table 8: Shared LLM configuration required for B–E. Blank values await the confirmatory freeze; historical settings do not fill these cells.

Field	Frozen value	Required handling
Provider / endpoint / immutable model version		Record resolved version; an alias alone is insufficient.
Temperature by call class		Match corresponding call classes across arms.
Top- p / presence and frequency penalties		Explicit values or documented provider defaults.
Maximum output tokens per attempt		Shared cap; record truncation and failures.
Maximum attempted calls / total tokens		Include all classes and retries.
Timeout / retry count / backoff		Fixed policy; record realized waits and failures.
Model-side seed / support status		Record support; do not promise service determinism.
Prompt corpus / template version		Shared information policy; prompt/reply hashes.
Plateau cadence / threshold / reset		Same trigger, including state-growth dependencies.

IPSM state-edge/code-gain ratios with zero code gain are undefined or shown as separate zero-gain cases.

Episodes within a run are dependent; uncertainty is clustered by run and reported by target/phase. Posterior trajectories and state weights are diagnostic, not evidence that every state is accurately learned. Correlation between IPSM growth and later code gain is evaluated within target and phase, without pooling raw counts across unlike programs.

6.6. Coverage and statistical analysis

Primary coverage outcomes are code-branch AUC over 24 hours and final code-branch count. AUC uses each run’s cumulative time series with a common interpolation convention and instrumentation, in branch-hours. Native endpoints at unequal durations are not substituted. Time to 95% of a target-specific common reference endpoint uses a threshold frozen before formal comparisons; unreached thresholds are censored.

Per target, report mean, median, dispersion, bootstrap 95% intervals, two-sided Mann–Whitney U tests, and Cliff’s δ . The primary coverage family has 36 tests: nine targets, D–C and E–D, final branches and AUC. Benjamini–Hochberg controls this family at $q = 0.05$. The primary admission-mechanism family has nine D–C productivity contrasts and is reported separately. Other diagnostics and sensitivity tests are exploratory. Effect direction is treatment minus comparator.

Cross-target summaries use equal-weight target effects and a hierarchical bootstrap resampling targets, then runs within targets. Raw branch counts from different programs

or individual executions are not pooled as independent samples. Target regressions and uncertainty accompany aggregates. Historical time-to-trigger uses Kaplan–Meier estimates, right censoring, and CVE-specific deadline recall. Fuzzer-failure interruptions are distinguished because censoring may be informative.

7. Available Evidence and Results

The empirical source for this revision is the supplied **Key_Experiment** directory. Earlier selected-run tables, significance counts, auxiliary comparisons, crash-class totals, and curves are not treated as measurements of the new controller. A blank numeric cell below means unavailable or unvalidated evidence, never a measured zero.

7.1. Archive inventory and observation horizon

The current extraction contains 94 AFLNet (A), 101 ChatAFL (B), and 108 benchmark LoopFuzz (D) summary rows, alongside 90 direct-admission (C) and 76 calibrated (E) ablation rows. The two sensitivity groups contain 24 rows each. Extra benchmark rows are retained as archive observations without assuming independence; groups with fewer than ten available runs retain a nominal count of ten, while the observed count n remains explicit. Means and sample SDs use observed values only; unavailable runs are left unfilled.

Benchmark LoopFuzz summaries include completed and interrupted rows; status counts are shown per target in Table 9. The exit-code labels do not independently establish

out-of-memory termination or a vulnerability. Runtime is fuzzer runtime, not total CPU time or necessarily the full startup-to-cleanup campaign duration.

The data mapping uses benchmark LoopFuzz as D (the supplied equivalent of `ablation/gated_fixed`), `ablation/direct` as C, and `ablation/calibrated` as E; the D source is counted once; two additional groups record $\gamma=0.99$ and $\gamma=1.0$ sensitivity runs. The startup logging-order issue in Table 4 means these declarations require launcher/end-record corroboration, and heterogeneous run counts prevent a complete matched-arm claim. No calibrated-arm effect is inferred from posterior fields alone.

Table 9: Supplied benchmark archive inventory. A/B/D use all available benchmark rows; the nominal comparison target is 10 runs per target. LoopFuzz status labels and exit codes are descriptive; exit code 137 is not independent evidence of an out-of-memory failure.

Target	AFLNet	LoopFuzz(D)	Completed	SIGABRT	Exit 137	Duration (min)
LightFTP	13	15	15	0	0	1473–1522
bftpd	10	20	13	0	7	1499–1510
ProFTPD	10	10	10	0	0	1489–1512
Pure-FTPD	11	10	10	0	0	1460–1472
Exim	10	10	10	0	0	1477–1486
Live555	10	10	10	0	0	1501–1505
Kamailio	10	11	11	0	0	1472–1518
Forked-daapd	10	12	12	0	0	1474–1524
Lighttpd1	10	10	10	0	0	1469–1523

Table 10: Exploratory native terminal endpoints (mean $\pm$ sample SD), retaining all available benchmark summary rows, including early terminations. These values are not matched-deadline or causal A–E comparisons.

Target	Code branches (b_abs)		IPSM state edges	
	AFLNet	Recorded LoopFuzz	AFLNet	Recorded LoopFuzz
LightFTP	71.0 $\pm$ 0.0	71.7 $\pm$ 2.3	187.3 $\pm$ 11.3	207.0 $\pm$ 17.0
bftpd	486.0 $\pm$ 8.8	502.7 $\pm$ 7.3	163.1 $\pm$ 15.4	226.7 $\pm$ 13.6
ProFTPD	5053.8 $\pm$ 97.4	5404.6 $\pm$ 95.6	190.5 $\pm$ 27.9	307.4 $\pm$ 18.6
Pure-FTPD	1076.4 $\pm$ 89.6	1223.3 $\pm$ 29.7	236.5 $\pm$ 26.7	321.4 $\pm$ 11.1
Exim	3798.8 $\pm$ 43.3	3914.9 $\pm$ 20.9	88.8 $\pm$ 5.2	152.7 $\pm$ 9.3
Live555	2893.6 $\pm$ 14.5	3057.9 $\pm$ 22.0	96.1 $\pm$ 8.6	171.9 $\pm$ 11.3
Kamailio	10002.6 $\pm$ 144.2	10764.5 $\pm$ 108.7	109.4 $\pm$ 9.7	134.9 $\pm$ 14.0
Forked-daapd	2330.8 $\pm$ 60.0	2670.2 $\pm$ 87.3	21.6 $\pm$ 2.1	22.8 $\pm$ 2.9
Lighttpd1	1940.7 $\pm$ 21.2	2127.9 $\pm$ 16.0	17.7 $\pm$ 2.5	28.8 $\pm$ 2.8

Table 11: Observed schema-v2 logging counts from the available benchmark LoopFuzz D archives. Reject, provisional, and durable are recorded labels, not verified post-execution queue membership.

Target	Candidates	Trials	Reject	Provisional	Durable	Episodes
LightFTP	48	48	44	0	4	13604
bftpd	26	26	26	0	0	6498
ProFTPD	0	0	0	0	0	1911
Pure-FTPD	173	173	166	0	7	5378
Exim	24	24	24	0	0	2650
Live555	778	778	771	0	7	7128
Kamailio	127	127	123	0	4	7038
Forked-daapd	412	411	410	0	1	12056
Lighttpd1	404	404	404	0	0	19368

Table 12: Archive-derived final code-branch endpoints and branch-coverage AUC. Values are mean $\pm$ sample SD over available runs; (n) is the observed run count and the nominal target is 10. AUC integrates the recorded `cov_over_time.csv` over each observed runtime; missing trajectories remain blank. These are descriptive archive results and do not imply matched-horizon causal effects.

Arm	Target	Runs	Final branches	Branch AUC (h)	IPSM state edges	Status
AFLNet	LightFTP	13/10	71.0 $\pm$ 0.0 (n=13)	1758.5 $\pm$ 33.8 (n=13)	187.3 $\pm$ 11.3 (n=13)	completed:13
AFLNet	bftpd	10/10	486.0 $\pm$ 8.8 (n=10)	11868.7 $\pm$ 165.6 (n=10)	163.1 $\pm$ 15.4 (n=10)	completed:10
AFLNet	ProFTPD	10/10	5053.8 $\pm$ 97.4 (n=10)	122652.5 $\pm$ 1730.1 (n=10)	190.5 $\pm$ 27.9 (n=10)	completed:10
AFLNet	Pure-FTPd	11/10	1076.4 $\pm$ 89.6 (n=11)	23054.1 $\pm$ 2084.4 (n=11)	236.5 $\pm$ 26.7 (n=11)	completed:11
AFLNet	Exim	10/10	3798.8 $\pm$ 43.3 (n=10)	91083.0 $\pm$ 1031.5 (n=10)	88.8 $\pm$ 5.2 (n=10)	completed:10
AFLNet	Live555	10/10	2893.6 $\pm$ 14.5 (n=10)	71127.4 $\pm$ 517.3 (n=10)	96.1 $\pm$ 8.6 (n=10)	completed:10
AFLNet	Kamailio	10/10	10002.6 $\pm$ 144.2 (n=10)	242853.7 $\pm$ 2772.6 (n=10)	109.4 $\pm$ 9.7 (n=10)	completed:10
AFLNet	Forked-daapd	10/10	2330.8 $\pm$ 60.0 (n=10)	54453.2 $\pm$ 1494.1 (n=10)	21.6 $\pm$ 2.1 (n=10)	completed:10
AFLNet	Lighttpd1	10/10	1940.7 $\pm$ 21.2 (n=10)	46909.4 $\pm$ 235.7 (n=10)	17.7 $\pm$ 2.5 (n=10)	completed:10
ChatAFL	LightFTP	11/10	71.3 $\pm$ 0.9 (n=11)	1728.1 $\pm$ 36.5 (n=11)	195.1 $\pm$ 14.6 (n=11)	completed:11
ChatAFL	bftpd	20/10	492.2 $\pm$ 13.7 (n=20)	12068.6 $\pm$ 315.2 (n=20)	167.1 $\pm$ 12.2 (n=20)	completed:20
ChatAFL	ProFTPD	10/10	5205.9 $\pm$ 166.7 (n=10)	125217.7 $\pm$ 2989.1 (n=10)	150.0 $\pm$ 18.5 (n=10)	completed:10
ChatAFL	Pure-FTPd	10/10	1109.3 $\pm$ 40.0 (n=10)	24976.6 $\pm$ 1126.8 (n=10)	186.0 $\pm$ 31.5 (n=10)	completed:10
ChatAFL	Exim	10/10	3761.7 $\pm$ 60.4 (n=10)	91301.7 $\pm$ 1501.4 (n=10)	115.8 $\pm$ 13.5 (n=10)	completed:10
ChatAFL	Live555	10/10	2882.4 $\pm$ 22.0 (n=10)	71224.8 $\pm$ 422.5 (n=10)	143.8 $\pm$ 8.8 (n=10)	completed:10
ChatAFL	Kamailio	10/10	10644.3 $\pm$ 96.5 (n=10)	257606.8 $\pm$ 2082.0 (n=10)	106.5 $\pm$ 6.8 (n=10)	completed:10
ChatAFL	Forked-daapd	10/10	2169.6 $\pm$ 113.0 (n=10)	49393.2 $\pm$ 3335.9 (n=10)	17.7 $\pm$ 4.9 (n=10)	completed:10
ChatAFL	Lighttpd1	10/10	1884.1 $\pm$ 25.3 (n=10)	45075.8 $\pm$ 729.3 (n=10)	14.3 $\pm$ 3.1 (n=10)	completed:10
LoopFuzz-direct	LightFTP	10/10	80.6 $\pm$ 15.5 (n=10)	1987.7 $\pm$ 394.1 (n=10)	210.6 $\pm$ 8.3 (n=10)	completed:10
LoopFuzz-direct	bftpd	10/10	498.9 $\pm$ 12.2 (n=10)	12082.0 $\pm$ 451.0 (n=10)	217.9 $\pm$ 13.8 (n=10)	completed:6; exit_137_near_limit:4
LoopFuzz-direct	ProFTPD	10/10	5446.3 $\pm$ 194.7 (n=10)	130538.5 $\pm$ 4511.6 (n=10)	300.7 $\pm$ 29.1 (n=10)	exit_137_near_limit:9; completed:1
LoopFuzz-direct	Pure-FTPd	10/10	1221.2 $\pm$ 48.9 (n=10)	28860.8 $\pm$ 1645.7 (n=10)	328.5 $\pm$ 22.5 (n=10)	completed:7; exit_137_near_limit:3
LoopFuzz-direct	Exim	10/10	3896.1 $\pm$ 25.6 (n=10)	94880.1 $\pm$ 928.3 (n=10)	150.9 $\pm$ 9.5 (n=10)	completed:10
LoopFuzz-direct	Live555	10/10	3075.7 $\pm$ 34.4 (n=10)	73511.0 $\pm$ 562.9 (n=10)	166.8 $\pm$ 9.3 (n=10)	completed:10
LoopFuzz-direct	Kamailio	10/10	10800.6 $\pm$ 129.0 (n=10)	260260.9 $\pm$ 2942.4 (n=10)	139.2 $\pm$ 9.8 (n=10)	completed:9; exit_137_near_limit:1
LoopFuzz-direct	Forked-daapd	10/10	2585.4 $\pm$ 86.2 (n=10)	61776.8 $\pm$ 1863.7 (n=10)	22.3 $\pm$ 2.2 (n=10)	completed:8; sigabrt:2
LoopFuzz-direct	Lighttpd1	10/10	2145.0 $\pm$ 69.8 (n=10)	53003.8 $\pm$ 1613.7 (n=10)	32.1 $\pm$ 5.0 (n=10)	completed:10
LoopFuzz-gated-fixed	LightFTP	15/10	71.7 $\pm$ 2.3 (n=15)	1780.7 $\pm$ 45.2 (n=15)	207.0 $\pm$ 17.0 (n=15)	completed:15
LoopFuzz-gated-fixed	bftpd	20/10	502.7 $\pm$ 7.3 (n=20)	12267.7 $\pm$ 377.5 (n=20)	226.7 $\pm$ 13.6 (n=20)	completed:13; exit_137_near_limit:7
LoopFuzz-gated-fixed	ProFTPD	10/10	5404.6 $\pm$ 95.6 (n=10)	132057.4 $\pm$ 2915.3 (n=10)	307.4 $\pm$ 18.6 (n=10)	completed:10
LoopFuzz-gated-fixed	Pure-FTPd	10/10	1223.3 $\pm$ 29.7 (n=10)	27863.9 $\pm$ 1496.9 (n=10)	321.4 $\pm$ 11.1 (n=10)	completed:10
LoopFuzz-gated-fixed	Exim	10/10	3914.9 $\pm$ 20.9 (n=10)	93659.0 $\pm$ 573.1 (n=10)	152.7 $\pm$ 9.3 (n=10)	completed:10
LoopFuzz-gated-fixed	Live555	10/10	3057.9 $\pm$ 22.0 (n=10)	75316.2 $\pm$ 723.5 (n=10)	171.9 $\pm$ 11.3 (n=10)	completed:10
LoopFuzz-gated-fixed	Kamailio	11/10	10764.5 $\pm$ 108.7 (n=11)	256813.5 $\pm$ 4223.4 (n=11)	134.9 $\pm$ 14.0 (n=11)	completed:11
LoopFuzz-gated-fixed	Forked-daapd	12/10	2670.2 $\pm$ 87.3 (n=12)	63646.7 $\pm$ 1977.0 (n=12)	22.8 $\pm$ 2.9 (n=12)	completed:12
LoopFuzz-gated-fixed	Lighttpd1	10/10	2127.9 $\pm$ 16.0 (n=10)	51220.4 $\pm$ 654.6 (n=10)	28.8 $\pm$ 2.8 (n=10)	completed:10
LoopFuzz-gated-calibrated	LightFTP	10/10	73.9 $\pm$ 4.5 (n=10)	1825.0 $\pm$ 107.6 (n=10)	210.9 $\pm$ 9.3 (n=10)	completed:10
LoopFuzz-gated-calibrated	ProFTPD	10/10	5469.8 $\pm$ 215.7 (n=10)	130564.7 $\pm$ 4604.1 (n=10)	307.8 $\pm$ 21.3 (n=10)	exit_137_near_limit:5; completed:5
LoopFuzz-gated-calibrated	Pure-FTPd	10/10	1215.4 $\pm$ 63.7 (n=10)	27894.3 $\pm$ 1737.6 (n=10)	311.6 $\pm$ 22.4 (n=10)	exit_137_near_limit:5; completed:5
LoopFuzz-gated-calibrated	Exim	10/10	3858.3 $\pm$ 32.8 (n=10)	93123.6 $\pm$ 924.6 (n=10)	138.6 $\pm$ 14.3 (n=10)	completed:10
LoopFuzz-gated-calibrated	Live555	10/10	3054.2 $\pm$ 20.5 (n=10)	72378.5 $\pm$ 1049.8 (n=10)	162.1 $\pm$ 4.4 (n=10)	completed:10
LoopFuzz-gated-calibrated	Kamailio	10/10	10830.0 $\pm$ 91.3 (n=10)	259377.2 $\pm$ 2649.6 (n=10)	130.9 $\pm$ 9.0 (n=10)	completed:10
LoopFuzz-gated-calibrated	Forked-daapd	6/10	2648.2 $\pm$ 116.0 (n=6)	62349.5 $\pm$ 1726.9 (n=6)	22.5 $\pm$ 2.1 (n=6)	completed:6
LoopFuzz-gated-calibrated	Lighttpd1	10/10	2068.1 $\pm$ 44.5 (n=10)	50385.1 $\pm$ 941.3 (n=10)	27.1 $\pm$ 4.4 (n=10)	completed:10
E, $\gamma = .99$	LightFTP	3/10	71.0 $\pm$ 0.0 (n=3)	1765.0 $\pm$ 17.8 (n=3)	203.7 $\pm$ 20.6 (n=3)	completed:3
E, $\gamma = .99$	bftpd	3/10	504.0 $\pm$ 8.2 (n=3)	12449.3 $\pm$ 107.5 (n=3)	220.3 $\pm$ 3.8 (n=3)	completed:1; oom_killed:2
E, $\gamma = .99$	ProFTPD	3/10	5520.3 $\pm$ 257.8 (n=3)	134617.2 $\pm$ 7078.5 (n=3)	296.0 $\pm$ 23.3 (n=3)	completed:2; oom_killed:1
E, $\gamma = .99$	Pure-FTPd	3/10	1229.0 $\pm$ 27.1 (n=3)	28031.6 $\pm$ 831.1 (n=3)	317.3 $\pm$ 19.0 (n=3)	completed:3
E, $\gamma = .99$	Exim	3/10	3877.7 $\pm$ 19.9 (n=3)	94524.9 $\pm$ 985.9 (n=3)	145.3 $\pm$ 6.7 (n=3)	completed:3
E, $\gamma = .99$	Live555	3/10	3137.7 $\pm$ 32.2 (n=3)	72830.7 $\pm$ 2052.5 (n=3)	164.0 $\pm$ 9.5 (n=3)	completed:3
E, $\gamma = .99$	Kamailio	3/10	10707.3 $\pm$ 342.2 (n=3)	235132.5 $\pm$ 53604.0 (n=3)	134.0 $\pm$ 5.6 (n=3)	completed:2; sigabrt:1
E, $\gamma = .99$	Lighttpd1	3/10	2086.7 $\pm$ 32.6 (n=3)	50671.9 $\pm$ 946.5 (n=3)	32.0 $\pm$ 9.0 (n=3)	completed:3
E, $\gamma = 1.0$	LightFTP	3/10	71.0 $\pm$ 0.0 (n=3)	1717.5 $\pm$ 89.2 (n=3)	218.3 $\pm$ 4.2 (n=3)	completed:3
E, $\gamma = 1.0$	bftpd	3/10	504.7 $\pm$ 6.1 (n=3)	12589.4 $\pm$ 115.3 (n=3)	227.0 $\pm$ 11.1 (n=3)	completed:2; oom_killed:1
E, $\gamma = 1.0$	ProFTPD	3/10	5300.7 $\pm$ 179.2 (n=3)	130892.4 $\pm$ 5230.8 (n=3)	290.0 $\pm$ 15.1 (n=3)	oom_killed:2; completed:1
E, $\gamma = 1.0$	Pure-FTPd	3/10	1176.0 $\pm$ 31.2 (n=3)	26176.9 $\pm$ 991.9 (n=3)	290.7 $\pm$ 16.3 (n=3)	oom_killed:1; completed:2
E, $\gamma = 1.0$	Exim	3/10	3852.0 $\pm$ 22.3 (n=3)	93809.3 $\pm$ 740.5 (n=3)	142.0 $\pm$ 11.1 (n=3)	completed:3
E, $\gamma = 1.0$	Live555	3/10	3111.0 $\pm$ 2.6 (n=3)	72199.6 $\pm$ 3600.0 (n=3)	165.0 $\pm$ 11.5 (n=3)	completed:2; sigabrt:1
E, $\gamma = 1.0$	Kamailio	3/10	10909.7 $\pm$ 72.5 (n=3)	264791.4 $\pm$ 2646.5 (n=3)	135.7 $\pm$ 8.1 (n=3)	completed:3
E, $\gamma = 1.0$	Lighttpd1	3/10	2034.0 $\pm$ 19.7 (n=3)	49632.1 $\pm$ 491.9 (n=3)	26.7 $\pm$ 1.2 (n=3)	completed:3

Table 10 retains every summary row, including interruptions. Its native terminal `b_abs` is taken from the supplied run summary; its IPSM state-edge count is likewise the supplied terminal summary value. The independently read trajectory for one D Lighttpd1 archive ends at 2,115 branches, whereas its summary records 2,139. The endpoint/cost tables retain the summary value and the trajectory plot retains the recorded time series; this unresolved source discrepancy is not silently reconciled. These observations are descriptive, not matched 24-hour outcomes. Sample standard deviations express variation among the supplied rows; neither causal claims nor significance tests are attached to unequal-duration endpoints.

7.2. RQ1: Current alignment observations

Forked-daapd has mean native code-branch endpoints of 2330.8 ± 60.0 for AFLNet and 2670.2 ± 87.3 for recorded LoopFuzz, while IPSM state-edge endpoints are 21.6 ± 2.1 and 22.8 ± 2.9 . This direction does not reproduce the older motivating claim of code regression. The difference cannot be attributed to calibration, which is not established as active.

Lighttpd1 increases in both native metrics: code branches from 1940.7 ± 21.2 to 2127.9 ± 16.0 , and IPSM state edges from 17.7 ± 2.5 to 28.8 ± 2.8 . These aggregate observations do not demonstrate the hypothesized small state expansion with hidden code progress. LightFTP has near-equal code endpoints, 71.0 ± 0.0 versus 71.7 ± 2.3 ,

with larger recorded LoopFuzz IPSM state-edge counts, 187.3 ± 11.3 versus 207.0 ± 17.0 . That pattern motivates a saturation analysis but does not establish the full time course or identify individual misleading states.

All three cases require common-build, common-horizon verification and event joins before becoming mechanism evidence. The available episode fields do support a narrower diagnostic: Fig. 4 compares the logged pre-update probabilities with the logged binary coverage-save reward, separately for each target. All 28,866 E-archive records pass finite-probability, Beta-mean, binary-reward, episode-ID uniqueness, chronological-order, and within-episode discounted-update checks. Excluding 5,753 zero-execution records leaves 23,113 records from 76 runs. Three between-episode posterior discontinuities are retained and documented; no missing history is reconstructed. The archived arm mapping does not by itself verify that every run used the intended scheduling policy.

Table 13 reports Brier score, ten-bin ECE, and AUPRC, together with a prequential reference that uses only past included rewards from the same run. Bootstrap resampling retains entire runs within each target; its intervals describe the supplied archive under that resampling scheme and do not resolve unknown dependence between launches. The saved reward counts native coverage-save events and can reflect hit-count novelty, and positive execution counts do not prove a complete fixed-energy episode. Consequently, these diagnostics do not fill the independent source-branch calibration outcomes in Table 15.

7.3. RQ2: Observed admission records and missing counterfactual

For the benchmark LoopFuzz D archive, the event stream contains 1,992 candidate records, 1,991 trial records, 1,968 reject labels, 23 durable labels, and no provisional labels. One candidate lacks a trial match; no trial lacks a candidate match. The corresponding C and E totals are 1,326/1,326 and 726/725 candidates/trials, respectively, with 1,326 direct durable labels for C and 716 rejects plus 9 durable labels for E. These are logged decisions rather than verified queue membership, and they do not demonstrate the required 99.9% event completeness.

Candidate/admission files occur in the C, D, and E archives with target-specific gaps; ProFTPD has no candidate/admission records in both D and E despite nonzero model-call counters. The supplied archives lack provisional and repair lifecycle files. Table 11 reports the D benchmark counts, while Table 12 gives the arm endpoints. Zero recorded lifecycle events do not yield a zero conversion rate or prove absence of repair/provisional behavior.

Disposition labels are not independently verified queue membership. The native-save ordering and state-only reachability problems prevent interpreting rejected labels as avoided queue pollution or durable labels as proof of strict predicate conformance. Without a validated C–D comparison and descendant lineage, downstream productivity,

E archives: pre-update probability versus logged save-event reward

Dots: episode-weighted frequency; whiskers: 95% bootstrap intervals across runs

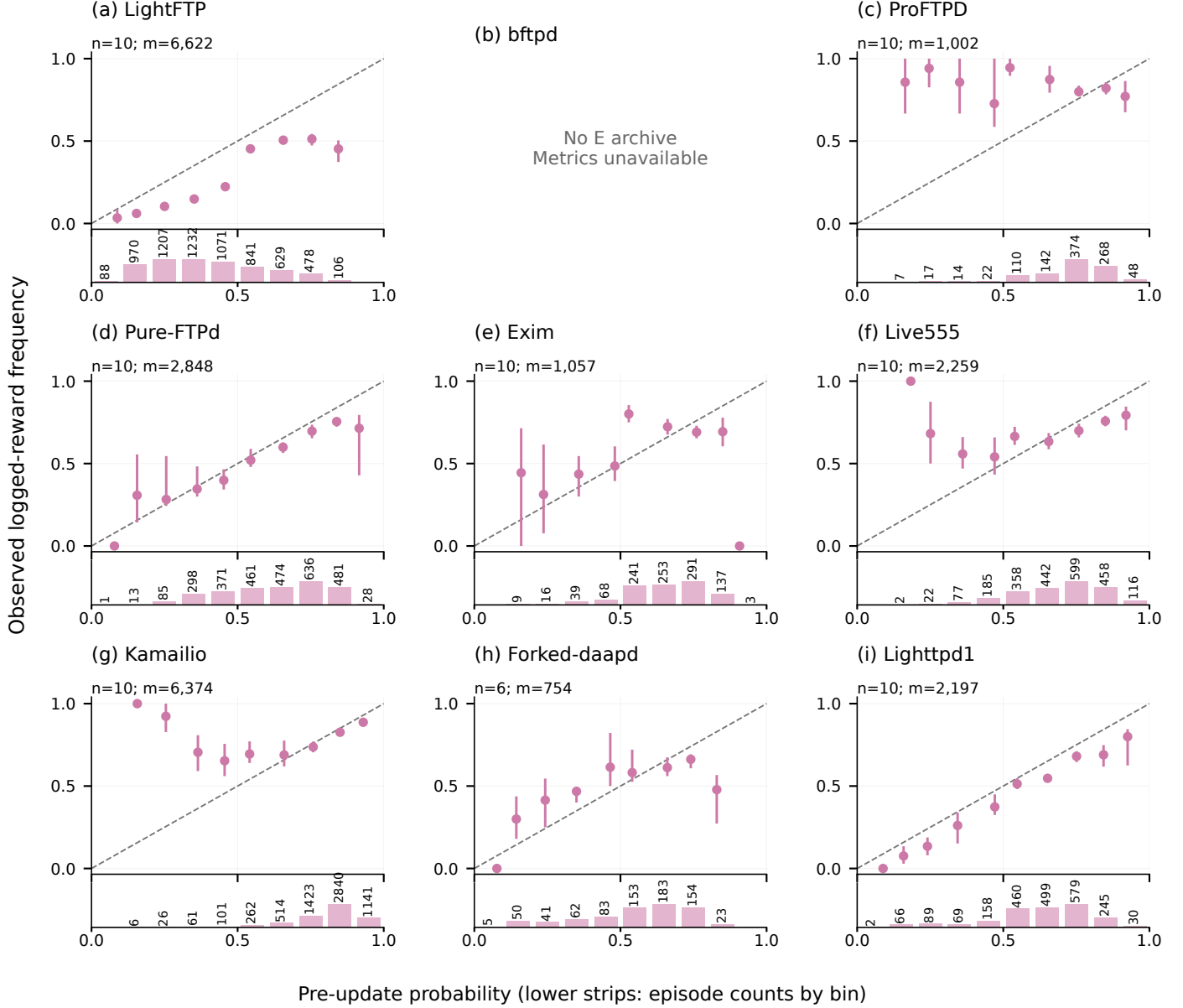


Figure 4: Descriptive reliability of the E-archive logged reward, shown separately by target. Pre-update probabilities are grouped into ten fixed equal-width bins; dots show the mean prediction and episode-weighted reward frequency. Whiskers are percentile 95% intervals from 2,000 resamples of whole runs within target; bins represented by fewer than two runs have no interval. Lower strips give exact episode counts; empty bins have no estimate. Only records with positive execution counts are included. The reward marks native coverage-save events, which may include hit-count novelty; it is not independently replayed source-branch gain. Episodes have unequal energy and no verified completion flag. Nominal $N = 10$; actual n and episode count m are shown.

Table 13: E-archive diagnostics for the logged coverage-save reward on positive-execution episodes. Values in brackets are percentile 95% intervals from 2,000 bootstrap resamples of whole runs within each target. Brier and ECE are lower-is-better; AUPRC uses non-interpolated average precision (AP). The prequential reference uses Beta(1,1)-smoothed past included rewards in the same run. These are descriptive diagnostics, not the fixed-energy code-branch metrics in Table 15.

Target	<i>n</i>	Episodes	Brier	Reference Brier	ECE (10 bins)	AUPRC (AP)
LightFTP	10	6,622	0.188 [0.178, 0.199]	0.183 [0.173, 0.196]	0.166 [0.162, 0.172]	0.452 [0.431, 0.471]
bftpd						
ProFTPD	10	1,002	0.183 [0.179, 0.188]	0.144 [0.126, 0.157]	0.137 [0.107, 0.173]	0.810 [0.782, 0.845]
Pure-FTPD	10	2,848	0.227 [0.217, 0.234]	0.243 [0.233, 0.248]	0.053 [0.042, 0.071]	0.700 [0.673, 0.721]
Exim	10	1,057	0.230 [0.221, 0.239]	0.219 [0.211, 0.227]	0.126 [0.105, 0.149]	0.703 [0.675, 0.734]
Live555	10	2,259	0.224 [0.212, 0.234]	0.218 [0.207, 0.229]	0.082 [0.065, 0.104]	0.742 [0.712, 0.772]
Kamailio	10	6,374	0.163 [0.154, 0.171]	0.161 [0.149, 0.172]	0.042 [0.032, 0.055]	0.858 [0.847, 0.866]
Forked-daapd	6	754	0.246 [0.236, 0.258]	0.252 [0.249, 0.256]	0.093 [0.063, 0.138]	0.622 [0.589, 0.644]
Lighttpd1	10	2,197	0.231 [0.225, 0.238]	0.251 [0.249, 0.253]	0.085 [0.073, 0.096]	0.665 [0.640, 0.687]

shadow-validation false negatives, and avoided nonproductive retention remain blank. Logged trial flags and latency can nevertheless be summarized descriptively in Table 15; Fig. 7 visualizes the reconciled record counts. A first-failure flag is a logged predicate result, not an independently validated reason for retention or rejection.

7.4. RQ3: Scheduling evidence and unfilled causal results

The supplied C, D, and E archives contain 56,164, 75,631, and 28,866 state-episode records, respectively. Their logged episode `mutations` counts range from zero to 18,816, which is not a shared fixed-energy measurement surface. Some coverage fields are named as code branches or code edges while the inspected implementation fills bitmap-slot counts or coverage-save-event counts. These values cannot be merged with source-level `b_abs` or treated as the main reward of Equation 4.

The archive contains E-arm calibrated runs and two small gamma sensitivity groups, but the logged implementation and heterogeneous horizons do not establish that all runs used the intended Thompson policy or that D–E is a matched causal comparison. Table 14 reports the recoverable endpoint/AUC values; cells without an E archive, such as bftpd, remain blank. Calibration benefits still require a frozen, matched-horizon analysis.

Fig. 5 displays the time-resolved branch observations over the first 24 hours, retaining per-time missingness. Its interpolation follows the linear convention used for the archive AUCs, while the table AUCs integrate each full observed trajectory. Changes in the set of contributing runs can change an aggregate curve even when individual histories do not decrease. In 121 of the 469 source trajectories, at least one recorded branch count decreases. The figure preserves these observations without applying a cumulative maximum or otherwise correcting the replay output; both source decreases and changing support can produce aggregate decreases.

Fig. 6 adds three reproducible state-level archive illustrations. For each motivating target, the selected E run

is the lower-median run by terminal branch count, with source path breaking ties; its two states with the most positive-execution episodes are shown. This rule uses no calibration score or effect direction. The complete logged histories for those states, including zero-execution episodes, expose the working posterior and realized scheduling multiplier. These cases illustrate recorded behavior and do not identify a causal response-state/branch mismatch.

7.5. RQ5: Recorded costs and interpretation limits

The final saved statistics across the benchmark Loop-Fuzz D archives report model calls and token counts by target in Table 16; the C/E arm ranges and means are retained in the machine-readable arm summary. Calls range from 52 to 248 per run in D. Thus the logged cap of 64 is not an enforced all-call campaign ceiling. These are saved tool counters, not independently reconciled provider billing. Some final aggregate statistics lag the event stream, so costs may omit the last reporting window.

Recorded configurations include a model alias, grammar/plateau temperatures of 0.5/1.2, top-*p* 1.0, maximum output tokens 4096, and total-token-cap field 0. The alias does not establish an immutable provider model. Startup configuration timing and missing build/resource provenance limit interpretation. These historical values do not fill the prospective shared configuration in Table 8.

Candidate token fields do not necessarily cover all call classes or retries; summing them is not total campaign cost. Low-activation targets, high-rejection targets, token outliers, saturation, and regressions must remain in the final analysis. Low activation may reflect productive mutation or a restrictive trigger; high rejection may reflect either useful gating or false negatives. Both need mechanism evidence.

Fig. 8 shows terminal coverage against saved token usage, and Fig. 9 shows the complementary saved-call view. Each target is plotted separately, with all available B–E observations retained. These views reveal dispersion and usage ranges without estimating a causal return per token

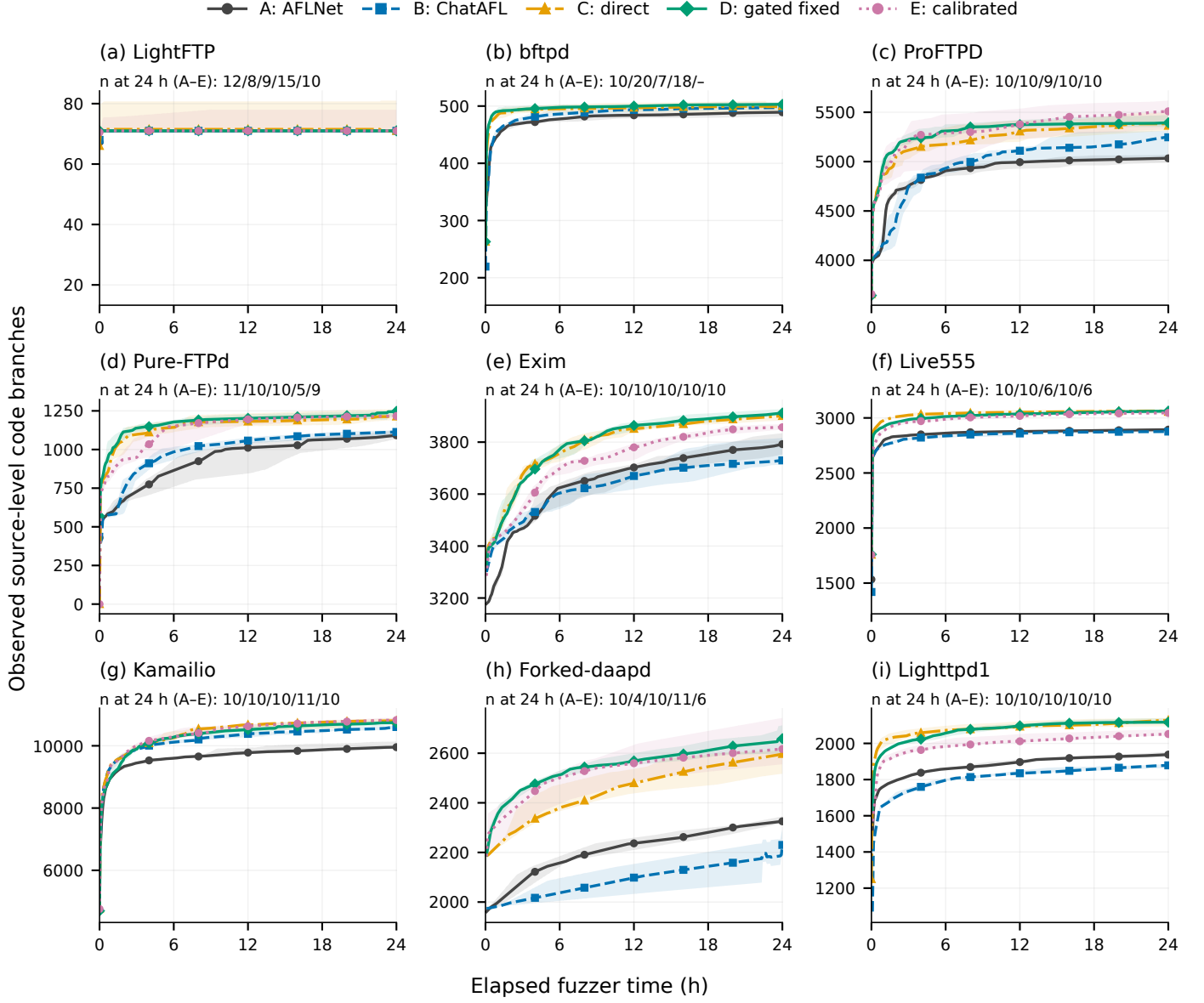


Figure 5: Observed A-E code-branch trajectories from 469 archives. Time is measured from each fuzzer_stats start time. Lines are medians and shaded regions are interquartile ranges, computed on a five-minute grid by linear interpolation only between recorded observations. A run contributes neither before its first observation nor after its last; the actual contributing n at 24 hours is shown in A-E order. The plot displays 0-24 hours without imposing a completed 24-hour campaign on interrupted runs. Full time-dependent denominators accompany the figure data; the nominal target remains $N = 10$. Missing E-bftpd is omitted. Native pipeline/build comparability remains unverified; these curves are descriptive, and shaded ranges are not confidence intervals.

Recorded posterior and scheduling multiplier: three E-archive cases
 Selection: lower-median terminal coverage run, then two most observed states

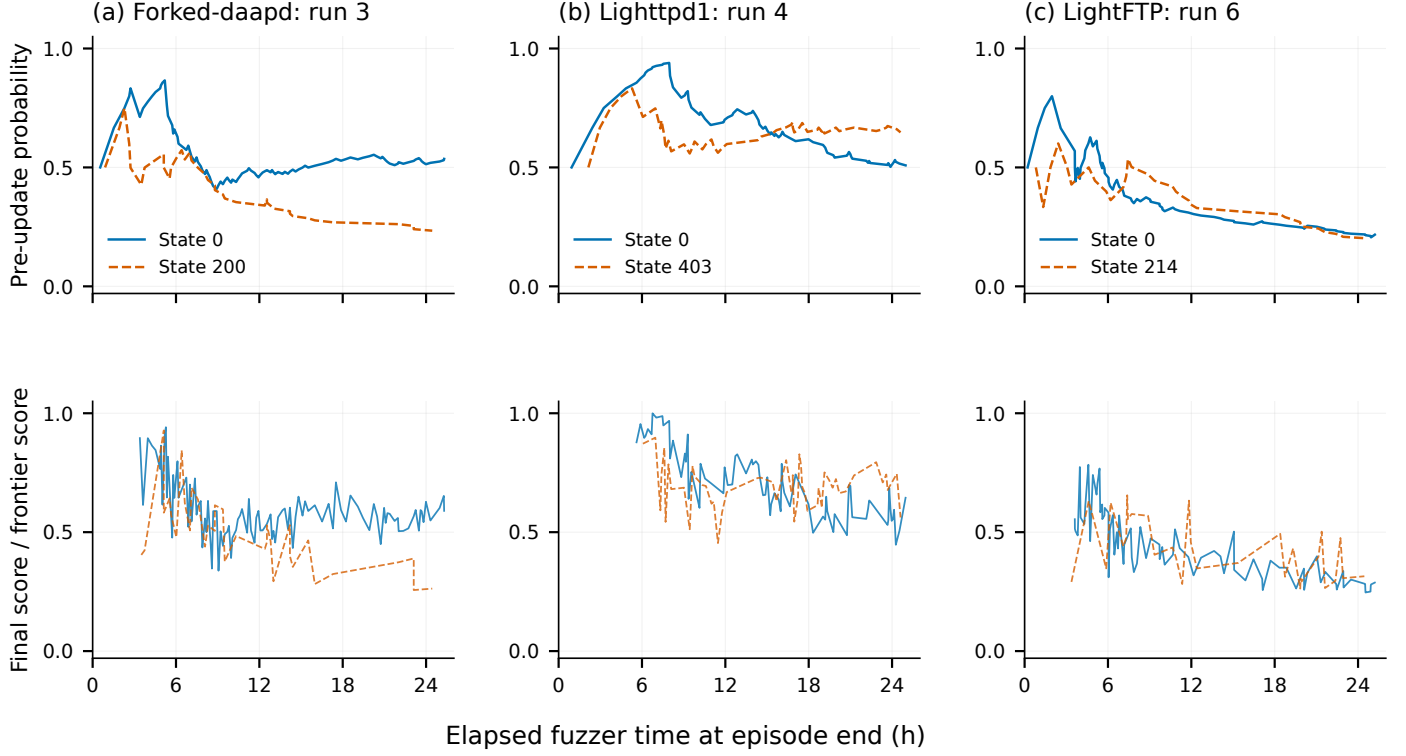


Figure 6: State-level E-archive illustrations for the three motivating targets, selected by the deterministic rule in the text. Upper panels show pre-update probabilities; lower panels show the logged final selection score divided by the frontier score. Each point is positioned at the logged episode-end time; connecting lines only guide the eye. Zero-frontier records have an undefined multiplier and are omitted from the lower panels. Colors identify states within each target, not equivalent states across targets. Unequal mutation energy, possible posterior resets, and native save-event rewards limit interpretation. The selected source paths and exact plotted values accompany the figure.

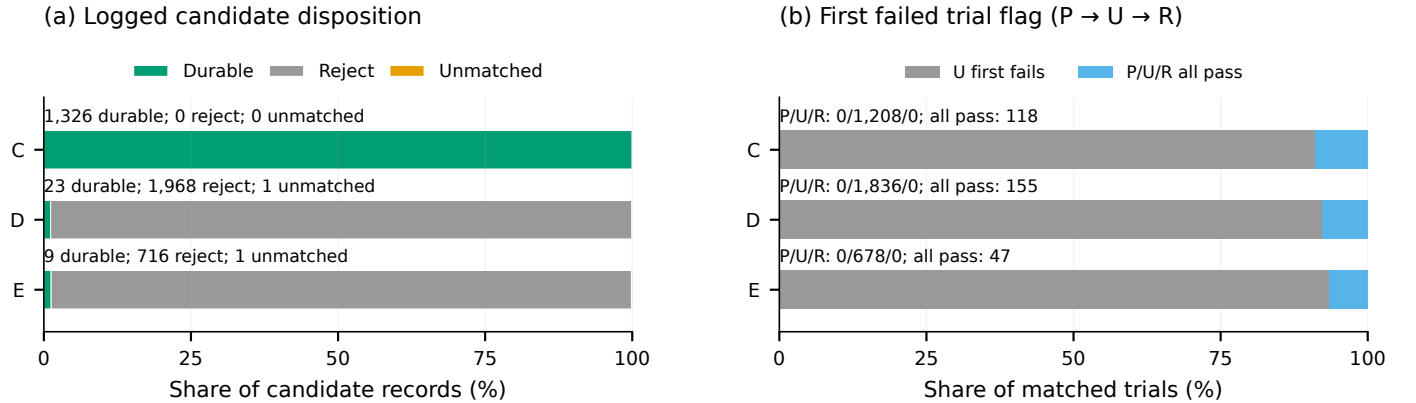


Figure 7: Logged candidate dispositions (left) and first failed trial flags in $P \rightarrow U \rightarrow R$ order (right), pooled within each archive arm. Exact counts accompany percentages. The candidate denominators are 1,326/1,992/726 for C/D/E; the trial denominators are 1,326/1,991/725. D and E each have one unmatched candidate, whose narrow segment is identified by the count. Passing $P/U/R$ does not establish code gain or durable queue membership. Provisional, converted, and expired outcomes have no validated lifecycle evidence and are not assigned zero rates.

Table 14: Archive-derived final code-branch count / branch-coverage AUC (branch-hours). Each cell shows mean $\pm$ sample SD, AUC on the second line, and the observed run count. The nominal target is 10 runs; missing target-arm archives remain blank. These descriptive endpoints do not replace matched-horizon causal inference.

Target	A	B	C	D	E
LightFTP	71.0 $\pm$ 0.0	71.3 $\pm$ 0.9	80.6 $\pm$ 15.5	71.7 $\pm$ 2.3	73.9 $\pm$ 4.5
	1758.5 $\pm$ 33.8	1728.1 $\pm$ 36.5	1987.7 $\pm$ 394.1	1780.7 $\pm$ 45.2	1825.0 $\pm$ 107.6
	n=13	n=11	n=10	n=15	n=10
bftpd	486.0 $\pm$ 8.8	492.2 $\pm$ 13.7	498.9 $\pm$ 12.2	502.7 $\pm$ 7.3	
	11868.7 $\pm$ 165.6	12068.6 $\pm$ 315.2	12082.0 $\pm$ 451.0	12267.7 $\pm$ 377.5	
	n=10	n=20	n=10	n=20	
ProFTPD	5053.8 $\pm$ 97.4	5205.9 $\pm$ 166.7	5446.3 $\pm$ 194.7	5404.6 $\pm$ 95.6	5469.8 $\pm$ 215.7
	122652.5 $\pm$ 1730.1	125217.7 $\pm$ 2989.1	130538.5 $\pm$ 4511.6	132057.4 $\pm$ 2915.3	130564.7 $\pm$ 4604.1
	n=10	n=10	n=10	n=10	n=10
Pure-FTPD	1076.4 $\pm$ 89.6	1109.3 $\pm$ 40.0	1221.2 $\pm$ 48.9	1223.3 $\pm$ 29.7	1215.4 $\pm$ 63.7
	23054.1 $\pm$ 2084.4	24976.6 $\pm$ 1126.8	28860.8 $\pm$ 1645.7	27863.9 $\pm$ 1496.9	27894.3 $\pm$ 1737.6
	n=11	n=10	n=10	n=10	n=10
Exim	3798.8 $\pm$ 43.3	3761.7 $\pm$ 60.4	3896.1 $\pm$ 25.6	3914.9 $\pm$ 20.9	3858.3 $\pm$ 32.8
	91083.0 $\pm$ 1031.5	91301.7 $\pm$ 1501.4	94880.1 $\pm$ 928.3	93659.0 $\pm$ 573.1	93123.6 $\pm$ 924.6
	n=10	n=10	n=10	n=10	n=10
Live555	2893.6 $\pm$ 14.5	2882.4 $\pm$ 22.0	3075.7 $\pm$ 34.4	3057.9 $\pm$ 22.0	3054.2 $\pm$ 20.5
	71127.4 $\pm$ 517.3	71224.8 $\pm$ 422.5	73511.0 $\pm$ 562.9	75316.2 $\pm$ 723.5	72378.5 $\pm$ 1049.8
	n=10	n=10	n=10	n=10	n=10
Kamailio	10002.6 $\pm$ 144.2	10644.3 $\pm$ 96.5	10800.6 $\pm$ 129.0	10764.5 $\pm$ 108.7	10830.0 $\pm$ 91.3
	242853.7 $\pm$ 2772.6	257606.8 $\pm$ 2082.0	260260.9 $\pm$ 2942.4	256813.5 $\pm$ 4223.4	259377.2 $\pm$ 2649.6
	n=10	n=10	n=10	n=11	n=10
Forked-daapd	2330.8 $\pm$ 60.0	2169.6 $\pm$ 113.0	2585.4 $\pm$ 86.2	2670.2 $\pm$ 87.3	2648.2 $\pm$ 116.0
	54453.2 $\pm$ 1494.1	49393.2 $\pm$ 3335.9	61776.8 $\pm$ 1863.7	63646.7 $\pm$ 1977.0	62349.5 $\pm$ 1726.9
	n=10	n=10	n=10	n=12	n=6
Lighttpd1	1940.7 $\pm$ 21.2	1884.1 $\pm$ 25.3	2145.0 $\pm$ 69.8	2127.9 $\pm$ 16.0	2068.1 $\pm$ 44.5
	46909.4 $\pm$ 235.7	45075.8 $\pm$ 729.3	53003.8 $\pm$ 1613.7	51220.4 $\pm$ 654.6	50385.1 $\pm$ 941.3
	n=10	n=10	n=10	n=10	n=10

or call. Measured CPU-hours and validated CVE recall are unavailable, so those cost-performance axes remain unreported.

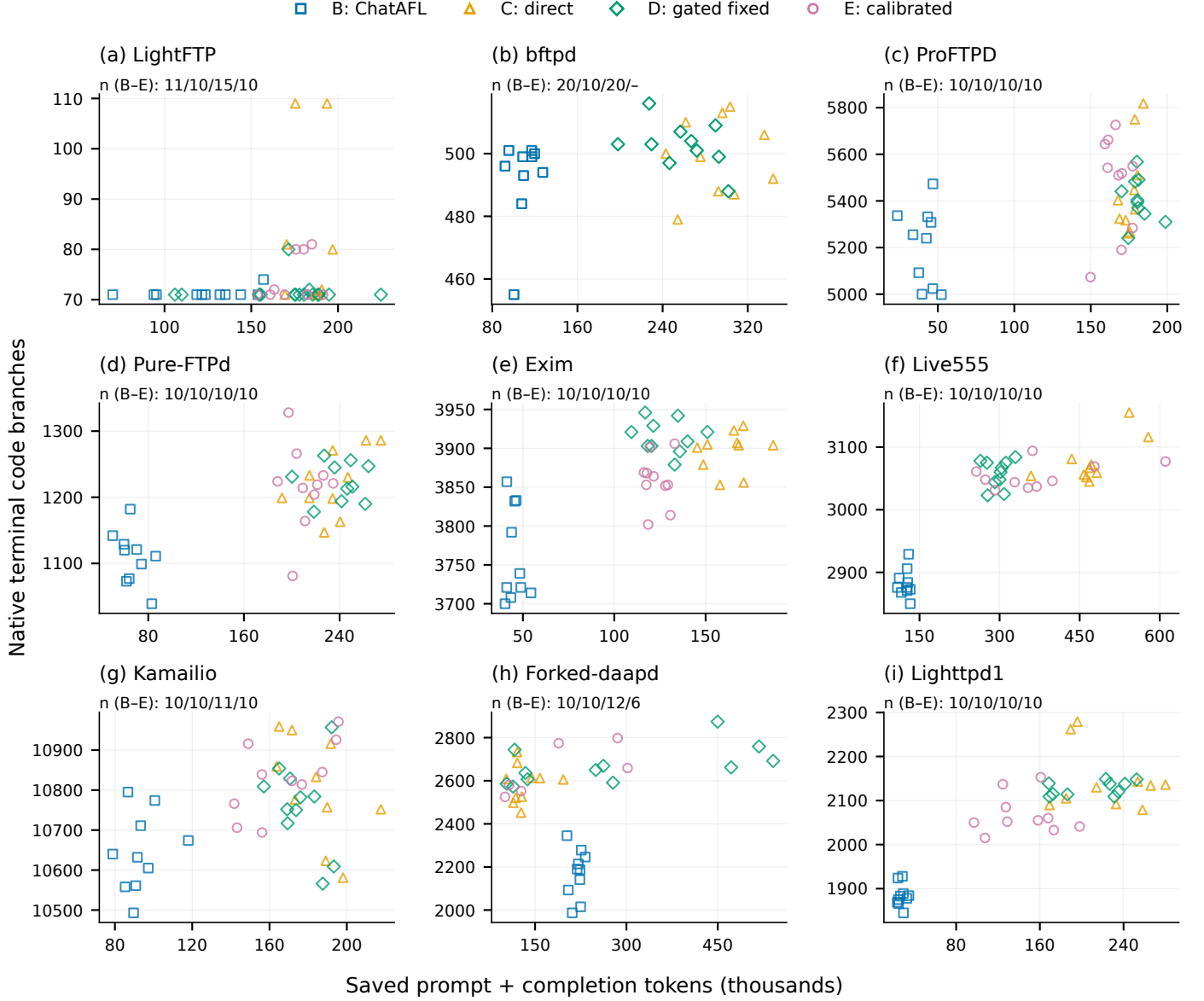


Figure 8: Saved token counters versus native terminal code coverage, with one marker per B-E archive (375 observations). Token usage is the saved prompt-plus-completion count, expressed in thousands; individual observations show the available dispersion without imposing a fitted efficiency frontier. Different targets have separate axes, and missing E-bftpd is omitted. AFLNet lacks a comparable saved counter and is not assigned a synthetic zero. Horizons and build provenance are not matched, and final counters may lag the event stream. This is a descriptive cost-coverage view, not reconciled billing or a cost-matched causal comparison.

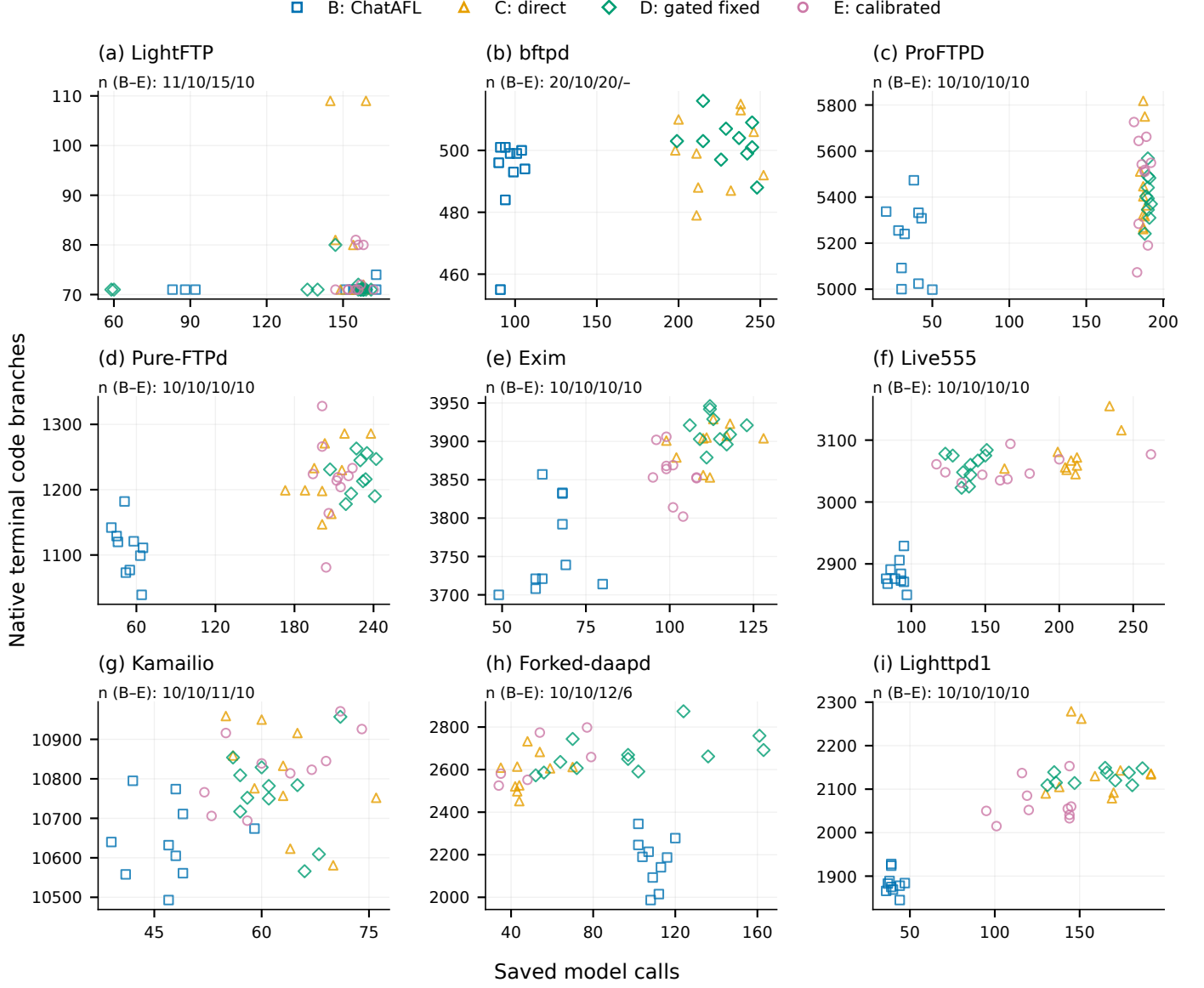


Figure 9: Saved model-call counters versus native terminal branch coverage for the same 375 B–E observations as Fig. 8. Each marker represents a run; no regression, Pareto frontier, or missing usage is inferred. Separate target scales preserve the different program coverage surfaces. Call counts do not measure token expenditure or CPU-hours.

Table 15: Logged mechanism totals across the available C/D/E archives. Candidate and trial IDs are scoped to each archive; disposition labels are not independently verified queue membership. Metrics requiring descendant joins or verified fixed-energy, independent code-branch rewards remain blank; the separate logged-reward diagnostics appear in Table 13.

Metric	C	D	E	Denominator / interpretation
Candidate records	1,326	1,992	726	Distinct candidate IDs within each archive.
Trial records	1,326	1,991	725	Distinct trial IDs within each archive.
Reject / durable labels	0 / 1,326	1,968 / 23	716 / 9	Logged trial dispositions.
First failed logged $P/U/R$	0 / 1,208 / 0	0 / 1,836 / 0	0 / 678 / 0	First false field in P , then U , then R order; trial denominator.
Direct durable rate	100.0% (1,326/1,326)	1.2% (23/1,992)	1.2% (9/726)	Logged disposition labels / generated candidates.
State-only provisional rate				All candidates; not enabled for C.
Provisional conversion / expiration				Provisional admissions, including pending/censored counts.
Trial latency (ms)	439.2±405.9 ($n = 67$)	381.9±561.3 ($n = 84$)	758.7±451.3 ($n = 55$)	Mean $\pm$ sample SD of within-run mean <code>latency_ms</code> ; runs with trials only.
Promotion latency				Requires verified queue-promotion timestamps.
Downstream productivity@64				Later descendants, excluding admission gain.
Queue size / nonproductive retention				Queue state plus C–D counterfactual.
Shadow-validation false-negative rate				Random rejected-candidate sample.
Code-branch gain per episode				Completed equal-energy episodes.
Brier / ECE / AUPRC				Fixed-energy source-branch reward; separate logged-reward diagnostics in Table 13.
Code gain per candidate / 1,000 tokens				Joined gain and complete usage.

Table 16: Recorded LoopFuzz D costs: per-run minimum–maximum across the available benchmark runs per target. Total tokens combine prompt and completion tokens; k denotes 1,000 tokens.

Target	Model calls	Total tokens (k)
LightFTP	59–161	106.0–224.7
bftpd	199–248	198.0–301.8
ProFTPD	188–192	170.1–198.8
Pure-FTPd	207–242	200.2–264.1
Exim	106–123	109.4–151.0
Live555	123–151	264.1–329.7
Kamailio	56–71	157.0–193.3
Forked-daapd	52–163	103.4–540.9
Lighttpd1	131–187	168.5–252.5

Table 17: Optional repair diagnostics. Repair is outside primary causal comparisons; unavailable observations stay blank.

Metric	E, repair off	E, repair on
Repair opportunities / attempts		
Before/after validation success		
Post-repair descendant code gain		
Changed fields / locality		
Token / latency cost		
Independent effect / interval		

8. Historical Vulnerability Rediscovery

Security outcomes use information-controlled rediscovery of historical vulnerabilities. Previously unknown vulnerabilities are not an optimization objective. Crash seeds, stack hashes, sanitizer messages, and protocol anomalies support triage; they are not independently counted vulnerabilities.

8.1. Vulnerable/patch-paired benchmark

For each vulnerability, construct a vulnerable base commit and the same base plus the minimal upstream security patch. Dependencies, flags, instrumentation, configuration, and input compatibility remain identical. Arbitrary old/new releases would confound the patch with unrelated changes.

Eligibility requires a documented historical vulnerability, an identifiable upstream patch, a network-protocol trigger, stable container builds, an independent oracle, compatible vulnerable/patched inputs, and no complex manual setup or exploit seed. Local-file-only or command-line-only bugs do not qualify.

The initial pilot contains six pairs across at least three protocols. Each pair must reach at least 95% oracle reproducibility in a declared number of validation attempts, show no corresponding patched-target activation, and support automatic build/reset/cleanup without information leakage. Expansion toward twelve follows only after these conditions hold. These are planned sizes, not completed CVEs; Table 18 contains no invented identifiers.

8.2. Information control and ground-truth oracles

CVE identifiers and security advisories are excluded from prompts and RFC corpora. Seeds contain no proof-of-concept exploit. The fuzzer cannot read patches, and benchmark metadata remains isolated. Where possible, a separate preparer assigns anonymous oracle IDs. This is information-controlled rediscovery, not fully blinded discovery: the preparer knows the vulnerability and model pretraining may include public information.

Reaching means visiting vulnerability-related code; triggering means the defined bug condition actually holds. Oracle priority is a source-level trigger predicate, sanitizer evidence, deterministic crash with patch validation, protocol semantic invariant, or reproducible vulnerable/patched behavioral difference. Covering a patched line is insufficient. Root-cause IDs deduplicate repeated triggers and crash manifestations.

Targets bind to localhost or isolated container networks, have no external network access, run without privileged containers, and use disposable state. Every trial resets the target. Release excludes unsanitized exploit corpora. These are benchmark requirements, not inferred properties of the supplied runs.

8.3. RQ4 outcomes and censoring

For each CVE and arm, report deadline rediscovery probability, time-to-trigger, reached-but-not-triggered outcomes, patched-target activation, and unique oracle/root-cause IDs. Record code coverage, executions, and tokens before the first trigger, and report CVEs per CPU-hour and 100,000 tokens. The formal rediscovery plan uses ten runs per arm with a declared 12-hour deadline frozen before execution.

Runs without triggers are right-censored at the observed deadline; interruptions are distinguished from ordinary deadline censoring. Kaplan–Meier curves are CVE-specific or explicitly stratified. Unlike vulnerabilities are not naively pooled as identical events. No normalized vulnerable/patch-paired result or validated trigger-time ledger is established by the current archive. A Kaplan–Meier estimate and its risk set therefore cannot be computed; no survival plot is shown. Crash-file counts are not substituted for rediscovery events, and Table 19 retains the missing outcomes.

9. Discussion

9.1. What calibration can and cannot repair

The controller calibrates the instrumental utility of an observed state proxy; it does not recover or verify semantic protocol state. If one response identifier aliases several server states, a single productivity estimate averages them. Reweighting the identifier cannot reconstruct lost distinctions. Richer parsing or internal-state instrumentation addresses different aspects at the cost of adapter complexity and deployment effort.

Low productivity can arise from a poor response proxy, saturated coverage, restrictive grammar, inadequate seeds, or insufficient energy. Independent reward prevents IPSM novelty from certifying itself but does not automatically diagnose these causes. The boundary analysis includes aliasing, cold start, phase, and target-specific regressions.

A fixed provisional horizon can discard a bridge whose payoff takes more descendants. Shadow validation and horizon sensitivity estimate this limitation; they do not prove every expired candidate worthless. Similarly, U/R can exclude useful error-handling inputs. Independent oracle retention and false-negative analysis are therefore necessary.

9.2. Cost, nonstationarity, and provider variability

Selection-indexed discounting can retain stale evidence for rarely selected states. Priors and the exploration floor preserve opportunities without guaranteeing fast identification. When code coverage saturates, nearly every reward may be zero, and calibration may offer little benefit.

Provider nondeterminism persists despite recorded prompts and fixed parameters. Aliases may resolve to different backends, retries alter costs, and target responses alter subsequent prompts. Equal-policy/equal-cap controls declared resources; actual usage and cost-matched analysis

Table 18: Historical vulnerability benchmark template. Six initial slots are reserved; identity, revision, oracle, and validation fields await evidence. Expansion toward twelve is conditional on pilot stability.

Slot	CVE / protocol	Vulnerable base	Security patch	Oracle / root cause	Repeatability	Patched activation
1						
2						
3						
4						
5						
6						

Table 19: Historical rediscovery outcomes, to be reported by CVE and arm. Blank cells indicate missing validated vulnerable/patch-paired evidence.

Outcome	A	B	C	D	E
CVE-level recall by deadline					
Time-to-trigger / uncertainty					
Reached but not triggered					
Patched oracle activation					
Executions / code coverage before trigger					
Tokens / CPU-hours before trigger					
Recall per CPU-hour / 100,000 tokens					

expose remaining tradeoffs. Provider-free replay audits the local controller rather than reproducing an adaptive campaign.

9.3. Decision criteria for unfinished experiments

Calibration remains a principal empirical claim only if the pilot produces nondegenerate predictions, reduces investment in branch-unproductive Forked-daapd states when observed, does not suppress Lighttpd1 solely for limited IPSM novelty, avoids severe throughput regressions on at least three pilot targets, and improves predictive diagnostics relative to a shared prequential base-rate predictor. D’s logged shadow productivity estimates provide an additional diagnostic; its raw queue-yield score is not treated as a calibrated probability. E–D differences require a plausible measured mechanism. These are development decisions, not confirmatory significance tests.

If these checks fail, calibration becomes exploratory and the empirical focus shifts to admission. If rediscovery does not improve, the result is reported without selecting easier CVEs after inspecting formal outcomes. Unsupported repair remains diagnostic. After raw-data and parameter freeze, revisions are limited to factual corrections, analysis/figure corrections, scoped confirmatory reruns, and

artifact documentation; they do not introduce new algorithms or tune formal results into success.

The planned full evidence package requires clear D–C and E–D estimates, complete candidate-level mechanisms, at least nine targets, twenty runs per core arm, and a stable historical benchmark of approximately eight to twelve vulnerabilities. These are completion criteria, not achieved sample counts. The current partial draft does not satisfy them.

10. Threats to Validity

The audit is retrospective. The inspected source may differ from archived executables; a recorded short commit string alone does not verify binary identity. Source-level concerns and archive-level facts retain separate provenance. An absence of corrected significance does not establish equivalence, while favorable descriptive endpoints do not establish a mechanism under unresolved duration, resource, or implementation differences. Admission, calibration, and rediscovery each require their own evidence.

Table 20: Threats, protocol controls, and remaining evidence limits.

Dimension	Required control	Current limitation
Internal	Admission-only C-D; scheduling-only E-D; repair off; paired snapshot diagnostics.	Missing target-arm observations and implementation-ordering differences prevent attribution.
Construct	Separate IPSM state edges, code branches, bitmap counts, reached/triggered events.	Native novelty and terminal summaries do not fully instantiate the proposed measures.
Statistical	Repeated trials, test families, effect sizes, clustered/hierarchical intervals, censoring.	Unequal-duration partial data cannot establish confirmatory effects.
Selection	Complete intention-to-run ledger and predefined infrastructure reruns.	Result manifests omit planned/unlaunched attempts; extra rows do not establish independent launches.
Reproducibility	Build/image pins, seeds, prompts/replies, event joins, resource traces.	Aliases and incomplete provenance prevent exact replication claims.
External	Multiple targets/protocols, negative cases, declared adapter assumptions.	Textual targets do not establish transfer to encrypted or opaque binary protocols.
Security	Minimal patch pairs, independent triggers, information separation.	No validated CVE matrix; model pretraining leakage cannot be excluded.

11. Conclusion

LoopFuzz treats response-derived protocol states and LLM output as uncertain evidence for resource allocation. Its controller contract separates code progress from IPSM novelty, gives state-only proposals bounded provisional validation, and calibrates state productivity at scheduling episodes. The response-visible interface remains useful, while independent code feedback determines its measured value.

The supplied benchmark and ablation archives support descriptive A-E endpoints, AUCs, time-resolved branch curves, saved-cost comparisons, and logged reward/admission diagnostics, together with a partial implementation audit. They do not yet establish causal admission gains, calibrated-scheduling gains, or improved historical rediscovery. The five-arm protocol, event schema, and blank results identify what remains to be measured. Effectiveness claims must follow those measurements, including unfavorable targets and resource costs.

CRedit authorship contribution statement

Author contribution roles remain to be confirmed by all authors before submission.

Declaration of competing interest

The competing-interest declaration remains to be confirmed by all authors.

Funding

Funding information remains to be confirmed by all authors.

Data availability

This revision uses the locally supplied experimental archive and includes a source-to-result audit in its companion materials. Public release location, access conditions, and permanent identifiers remain to be confirmed. The intended package contains build manifests, launch ledgers, raw event schemas, sanitized prompts/replies, analysis scripts, and patch-pair oracle metadata with information separation. No public archive or exact provider replay is claimed.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

OpenAI Codex assisted manuscript restructuring, language revision, LaTeX editing, figure-generation and analysis scripts, and consistency checks. The authors must verify the technical content, references, empirical interpretations, and final disclosure before submission.

References

- [1] V.-T. Pham, M. Böhme, A. Roychoudhury, AFLNet: A greybox fuzzer for network protocols, in: 2020 IEEE 13th International Conference on Software Testing, Verification and Validation (ICST), 2020, pp. 460–465. doi:10.1109/ICST46399.2020.00062.
- [2] R. Natella, StateAFL: Greybox fuzzing for stateful network servers, Empirical Software Engineering 27 (2022). doi:10.1007/s10664-022-10233-3, art. no. 191.
- [3] R. Meng, M. Mirchev, M. Böhme, A. Roychoudhury, Large language model guided protocol fuzzing, in: 31st Annual Network and Distributed System Security Symposium (NDSS 2024), 2024, pp. 1–17. doi:10.14722/ndss.2024.24556.
- [4] S. Qin, F. Hu, Z. Ma, B. Zhao, T. Yin, C. Zhang, NSFuzz: Towards efficient and state-aware network service fuzzing, ACM Transactions on Software Engineering and Methodology 32 (2023) 160:1–160:26. doi:10.1145/3580598.
- [5] J. Ba, M. Böhme, Z. Mirzamomen, A. Roychoudhury, Stateful greybox fuzzing, in: 31st USENIX Security Symposium (USENIX Security 2022), 2022, pp. 3255–3272.

- [6] C. S. Xia, M. Paltenghi, J. L. Tian, M. Pradel, L. Zhang, Fuzz4All: Universal fuzzing with large language models, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, 2024, pp. 1–13. doi:10.1145/3597503.3639121.
- [7] J. Wang, L. Yu, X. Luo, LLMIF: Augmented large language model for fuzzing IoT devices, in: 2024 IEEE Symposium on Security and Privacy (SP), 2024, pp. 881–896. doi:10.1109/sp54263.2024.00211.
- [8] P. Srivastava, M. Payer, Gramatron: effective grammar-aware fuzzing, in: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2021, pp. 244–256. doi:10.1145/3460319.3464814.
- [9] D. Wang, Y. Li, Z. Zhang, K. Chen, CarpetFuzz: Automatic program option constraint extraction from documentation for fuzzing, in: 32nd USENIX Security Symposium (USENIX Security 2023), 2023, pp. 1919–1936.
- [10] R. Natella, V.-T. Pham, ProFuzzBench: A benchmark for stateful protocol fuzzing, in: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2021, pp. 662–665. doi:10.1145/3460319.3469077.
- [11] M. Schloegel, N. Bars, N. Schiller, L. Bernhard, T. Scharnowski, A. Crump, A. Ale-Ebrahim, N. Bissantz, M. Muench, T. Holz, SoK: Prudent evaluation practices for fuzzing, in: 2024 IEEE Symposium on Security and Privacy (SP), 2024, pp. 1974–1993. doi:10.1109/sp54263.2024.00137.
- [12] M. Böhme, L. Szekeres, J. Metzman, On the reliability of coverage-based fuzzer benchmarking, in: Proceedings of the 44th International Conference on Software Engineering, 2022, pp. 1621–1633. doi:10.1145/3510003.3510230.